

PROJECT REPORT

on

CodeSageAI

from

Webcos Software Solutions

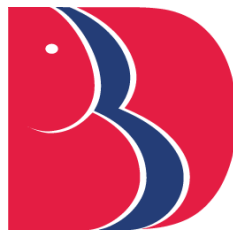
**Towards partial fulfillment of the requirements
for the award of degree of**

**Bachelor of Computer Applications
(Data Science & Artificial Intelligence)**

from

Babu Banarasi Das University

Lucknow



Academic Session 2025 – 26

Department of Data Science & Artificial Intelligence

School of Computer Applications

I Floor, CA- BLOCK, BBD University, BBD City, Ayodhya Road, Lucknow (U. P.) INDIA 226028

PHONE: HEAD: 0522-6196242 Dept. T&P Cell: 9990462250

www.bbdun.ac.in

PROJECT REPORT

on

CodeSageAI

from

Webcos Software Solutions

**Towards partial fulfillment of the requirements
for the award of degree of**

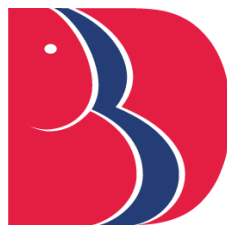
Bachelor of Computer Applications

(Data Science & Artificial Intelligence)

from

Babu Banarasi Das University

Lucknow



**Submitted by
Pradyumna Singh
1230258295**

**Under Guidance of
Mr. Akshay Mishra
Assistant Prof.**

Academic Session 2025 – 26

Department of Data Science & Artificial Intelligence

School of Computer Applications

I Floor, CA- BLOCK, BBD University, BBD City, Ayodhya Road, Lucknow (U. P.) INDIA 226028

PHONE: HEAD: 0522-6196242 Dept. T&P Cell: 9990462250

w w w . b b d u . a c . i n

UNDER TAKING

This is to certify that Project Report entitled

CodeSageAI

being submitted by

Pradyumna Singh

Towards the partial fulfillment of the requirements

for the award of the degree of

Bachelor of Computer Applications
(Data Science & Artificial Intelligence)

from

Babu Banarasi Das University

Lucknow



Academic Year 2025-26

is a record of the student's own work carried out at

Webcos Software Solutions

and to the best of our knowledge the work reported herein does not form a part of any other thesis or work on the basis of which degree or award was conferred on an earlier occasion to this or any other candidate.

Authorized Signatory

Webcos Software Solutions

Students Signature

Pradyumna singh

1230258295



Certificate of Excellence

Presented to :

Mr. Pradyumna Singh Bhadoria

Technology: Python+Django & Generative AI

"Has successfully completed the requirements to be recognized as a Webcos Certified IT Professional. The duration of the program was 90days."

WEBCOS
SOFTWARE SOLUTIONS



Enrollment No: WCS 203
Certificate issue date: 1-May-2026

Ayush Jaiswal
Chief Executive Officer
(CEO)



DECLARATION

I hereby declare that the report entitled “**CodeSageAI**”, submitted to the *Department of Data Science & Artificial Intelligence, School of Computer Applications, Babu Banarasi Das University*, BBD City, Ayodhya Road, Lucknow, Uttar Pradesh – 226028, is submitted in partial fulfilment of the requirements for the award of the degree of **Bachelor of Computer Applications (Data Science & Artificial Intelligence)**.

This report is the outcome of my own effort and has been completed under the guidance and supervision of **Mr. Akshay Mishra, Assistant Professor, Department of Data Science & Artificial Intelligence, School of Computer Applications, Babu Banarasi Das University, Lucknow**. I also declare that the contents of this report have not been submitted, either in full or in part, to any other university or institution for the award of any degree or diploma.

Place: Lucknow (UP)

Date:

Student Name: Pradyumna Singh

University Roll No.: 1230258295

Signature of the Student:

ACKNOWLEDGEMENT

The completion of this project marks not only an academic milestone but also a deeply personal journey filled with learning, challenges, and growth. I would like to express my heartfelt gratitude to all those who supported and guided me throughout this journey. The successful completion of this work would not have been possible without the encouragement and assistance of many individuals.

First and foremost, I would like to extend my profound gratitude to **Dr. Reena Srivastava**, Honorable **Dean**, School of Computer Applications, Babu Banarasi Das University for her continuous support, motivation, and encouragement. Her visionary leadership and guidance have always been a source of inspiration for us.

I am also deeply thankful to **Dr. Manish Kumar, Head of the Department**, Department of Data Science and Artificial Intelligence for his constant support and valuable guidance, which played a vital role in helping me stay focused and complete this project successfully.

I would like to express my sincere gratitude to **Mr. Sri Nivas Singh, T&P Coordinator**, for his support, cooperation, and coordination throughout the project.

I extend my deep appreciation to my project guide, **Mr. Akshay Mishra, Assistant Professor**, for his unwavering support, expert guidance, patience, and insightful suggestions. His continuous encouragement and feedback greatly contributed to the successful completion of this work and enhanced my understanding.

Last but not the least, I sincerely thank my **Family, Friends** especially Somya, and mentors for their constant support, encouragement, and motivation. Their belief in me has been a strong pillar behind my consistent efforts.

Table of Contents

S.No.	Title	Page No.
I.	Cover Page	-
II.	Certificate	-
III.	Declaration	-
IV.	Acknowledgement	-
V.	Title of Project	-
1.	Introduction	1-4
2.	Purpose & Scope	5-7
3.	Survey of Technology	8-11
4.	Requirement Analysis	12-16
5.	Problem Definition	17-18
6.	Planning & Scheduling	19-21
7.	Software Requirement Specification	22-26
8.	Software Engineering Paradigm Applied	27-28
9.	Module Descriptions	29-31
10.	Logical Design (DFDs)	32-36
11.	Database Design	37
12.	User Interface Design	38-41
13.	Coding	42-73
14.	Screenshots	74-78
15.	Testing & Implementation	79-81
16.	System Security Measures	82-83
17.	Cost Estimation	84-85
18.	Conclusion	86
19.	Future Scope & Enhancement	87-88
20.	Bibliography	89
21.	Glossary	90-91
22.	Appendices	92-94

TITLE OF THE PROJECT



"Understand Code. Learn Smarter."

CodeSageAI

INTRODUCTION OF THE PROJECT

Overview

In the modern digital era, programming has become a fundamental skill across various domains such as software development, data science, artificial intelligence, and automation. However, understanding programming logic, debugging errors, and writing efficient code remain major challenges, especially for beginners and students. Traditional learning methods, including textbooks, online tutorials, and documentation, often provide static and theoretical knowledge, which may not be sufficient for developing a deep understanding of coding concepts. As a result, learners frequently struggle to interpret complex code, identify errors, and improve performance, leading to a fragmented and inefficient learning process.

With the rapid advancement of Artificial Intelligence (AI) and Machine Learning (ML), intelligent systems are now capable of understanding patterns, processing natural language, and generating meaningful outputs. One of the most significant developments in this field is the emergence of Large Language Models (LLMs), which can analyze both natural language and programming code. These models are trained on vast datasets and can generate human-like explanations, making them highly effective for educational and problem-solving applications. The integration of AI and ML into software systems has opened new possibilities for creating intelligent learning platforms that provide real-time assistance and personalized guidance.

The proposed project, CodeSageAI, is an AI-powered intelligent code explanation and learning platform developed using Python and the Django framework. The system integrates Machine Learning techniques and an LLM-based conversational model to analyze source code and generate structured explanations in natural language. It is designed to act as a smart programming tutor that helps users understand the logic behind the code rather than just executing it. The frontend of the system is developed using HTML, CSS, JavaScript, Django Template Language (DTL), jQuery, and AJAX, ensuring a dynamic, responsive, and user-friendly interface. The backend handles user authentication, data management, and communication with the AI model, providing seamless interaction between the user and the system.

CodeSageAI addresses the gap between writing code and understanding it by offering features such as line-by-line explanation, error detection, optimization suggestions, and time and space complexity analysis. Unlike traditional systems that focus mainly on code execution, this platform emphasizes conceptual learning and clarity. Additionally, the system includes an AI-powered chatbot module that

allows users to ask programming-related questions and receive instant responses. This interactive approach enhances learning by providing continuous support and reducing the need to switch between multiple resources.

Furthermore, the system is designed to support multiple programming languages, making it accessible to a wide range of users. It also maintains user history, feedback, and reports, enabling tracking of learning progress and system performance. By combining AI, ML, and modern web technologies, CodeSageAI creates a comprehensive and intelligent learning environment that improves coding skills, enhances problem-solving abilities, and promotes independent learning.

In conclusion, CodeSageAI demonstrates the practical application of Artificial Intelligence and Machine Learning in the field of education. It transforms the traditional approach to learning programming by introducing an intelligent, interactive, and user-centric system. The project not only improves code comprehension but also contributes to the development of smarter educational tools, making programming more accessible, efficient, and engaging for learners.

Background and Motivation

The rapid growth of software development has created a strong demand for intelligent tools that help developers write efficient, error-free, and optimized code. With advancements in Artificial Intelligence and Large Language Models (LLMs), development environments are becoming more interactive and automated. Technologies such as Python and frameworks like Django have simplified application development, while AI tools such as GitHub Copilot and ChatGPT provide coding assistance. However, these tools mainly focus on code generation rather than structured analysis and learning. As a result, many developers-especially beginners-face challenges in understanding code logic, debugging errors, and improving performance. This highlights the need for a system that not only analyzes code but also provides clear, structured, and educational feedback.

The motivation behind CodeSageAI is to develop an intelligent system that functions as both a code analyzer and a learning assistant. It aims to support beginners by offering easy-to-understand explanations and structured insights, while also reducing the effort required for debugging through automated error detection and improvement suggestions. Additionally, the system focuses on performance optimization by providing time and space complexity analysis. By leveraging modern AI technologies such as Groq, CodeSageAI delivers fast, accurate, and human-like feedback. Ultimately, the project is motivated by the goal of bridging the gap between learning and development tools, creating a platform that enhances both coding skills and productivity.

Objectives

To develop an AI-powered system using Machine Learning and LLM models for intelligent understanding and explanation of programming code. This objective focuses on building a smart platform that can interpret source code using advanced AI techniques. The system leverages Machine Learning and Large Language Models to analyze code logic and generate meaningful, human-readable explanations.

To provide structured, line-by-line explanations and help beginners understand program logic in a simple and educational manner. The system aims to break down complex programs into smaller understandable parts. By presenting explanations in a structured and beginner-friendly format, it supports conceptual clarity and improves learning outcomes.

To detect syntax and logical errors, suggest optimized solutions, and analyze time and space complexity. This objective ensures the system not only explains code but also improves code quality. It identifies errors, provides correction suggestions, and evaluates performance factors such as efficiency and resource usage.

To generate test cases and support multiple programming languages for effective code validation and learning. The platform will create sample test cases to verify correctness of programs. Supporting multiple languages increases accessibility and makes the system useful for a wide range of learners.

To enhance programming education through an interactive AI chatbot, reducing dependency on multiple learning platforms. An AI-driven conversational assistant will provide real-time answers to programming queries. This feature promotes self-learning and minimizes the need to switch between various resources.

Utility of the Project

The CodeSageAI system provides significant benefits in the fields of education, software development, and intelligent learning systems. It acts as an AI-powered programming tutor that helps users understand complex programming logic in a simplified and structured manner.

The project is useful for:

- ***Students and Beginners:*** Helps them understand source code, programming concepts, and debugging techniques without depending on multiple resources.
- ***Educational Institutions:*** Can be integrated into e-learning platforms and labs to assist in programming education.

- ***Online Learning Platforms:*** Enhances learning experience by providing AI-generated explanations and interactive support.
- ***Developers:*** Assists in reviewing code, detecting errors, and optimizing performance.
- ***Self-Learners:*** Provides 24/7 AI support through chatbot interaction.
- ***Research and AI Applications:*** Demonstrates the practical use of Machine Learning and Large Language Models in real-time systems.

By reducing learning time, improving code comprehension, and providing structured assistance, the system contributes to modern AI-driven education.

PURPOSE & SCOPE

Purpose of the Project

The purpose of CodeSageAI is to design and develop an intelligent, AI-powered code analysis system that assists developers in writing high-quality, efficient, and error-free code. In today's fast-evolving software development environment, programmers often face challenges in debugging, understanding complex logic, and optimizing performance. CodeSageAI aims to address these challenges by providing a platform that can analyze user-submitted code and generate structured, meaningful feedback.

The system is built using technologies such as Python and the web framework Django, along with advanced AI capabilities powered by platforms like Groq. By integrating these technologies, CodeSageAI is capable of performing multiple tasks, including identifying syntax and logical errors, suggesting improvements, analyzing time and space complexity, and recommending best coding practices.

Another important purpose of the project is to support learning and skill development among students and beginner programmers. Unlike traditional tools that primarily focus on code execution or generation, CodeSageAI emphasizes explanation and understanding. It provides clear, well-structured insights into how the code works, what issues exist, and how they can be resolved. This helps users not only fix their code but also improve their problem-solving abilities and coding techniques.

Furthermore, CodeSageAI aims to reduce the time and effort required for manual debugging and code review by automating the analysis process. It enhances developer productivity by offering quick and reliable feedback, enabling users to focus more on logic building and innovation rather than repetitive error-checking tasks.

In addition, the project seeks to bridge the gap between development tools and educational platforms by combining automation with learning. It creates an interactive environment where users can upload their code, receive detailed analysis, and continuously improve their skills. Overall, the purpose of CodeSageAI is to provide a smart, efficient, and educational solution that supports both coding productivity and knowledge enhancement.

Scope of the Project

The scope of CodeSageAI encompasses the following functional areas:

1. In-Scope Features

Module	Description	Status
User Authentication	Email-based registration, login, logout, and secure session management using Django authentication system.	Implemented
Code Input System	Users can upload or directly enter code into the system for analysis.	Implemented
AI Code Analysis	AI-powered analysis of code to detect syntax errors, logical issues, and code quality using Groq API.	Implemented
Bug Detection	Identifies errors and potential issues in the submitted code with clear explanations.	Implemented
Code Improvement Suggestions	Provides recommendations to improve readability, efficiency, and maintainability.	Implemented
Time & Space Complexity	Analyzes code performance and provides time and space complexity insights.	Implemented
Structured Output Display	Displays results in a clean, formatted, and easy-to-understand structure.	Implemented
Multi-language Support	Supports multiple programming languages such as Python, C++, Java, and JavaScript.	Implemented
User-Friendly Interface	Simple and responsive UI built using HTML, CSS, JavaScript, and Django templates.	Implemented

2. Out-of-Scope (Future Enhancements)

- Integration with IDEs (e.g., VS Code, PyCharm)
- Real-time code analysis while typing
- Mobile application (Android/iOS)
- Voice-based code input and AI assistance
- Advanced debugging and execution environment
- Collaboration features (team-based coding analysis)
- Integration with version control systems (GitHub, GitLab)

Target Audience

The target audience for CodeSageAI includes:

- ***Students and Beginners:*** Individuals learning programming who need guidance and structured explanations.
- ***Developers:*** Programmers who want to analyze, debug, and optimize their code efficiently.
- ***Educators:*** Teachers and trainers who want tools to demonstrate code analysis and best practices.
- ***Self-learners:*** Individuals improving their coding skills through practice and feedback.

SURVEY OF TECHNOLOGY

Introduction to the Technology Stack

The CodeSageAI platform is developed using a modern and efficient technology stack designed to support intelligent code analysis and user interaction. Each technology has been selected based on its performance, scalability, and compatibility with AI-based systems. The project integrates web development frameworks, programming languages, and AI tools to create a seamless and responsive application. This chapter provides an overview of the technologies used and their role in the development of the system.

Backend Framework: Django

Django is a high-level web framework based on Python, which enables rapid development and clean, maintainable design. It follows the Model-View-Template (MVT) architecture and provides built-in features that simplify backend development.

Django is used in CodeSageAI for the following purposes:

- ***User Authentication System:*** Handles registration, login, logout, and session management
- ***Database Management:*** Uses ORM to interact with the database efficiently
- ***Security Features:*** Protection against common attacks such as CSRF and SQL injection
- ***Scalability:*** Supports structured and modular application development

The backend logic, request handling, and integration with AI APIs are managed using Django.

Frontend Technologies

1. HTML, CSS, and JavaScript

The frontend of CodeSageAI is built using standard web technologies including HTML, CSS, and JavaScript. These technologies are responsible for designing the user interface and ensuring smooth interaction between the user and the system.

- ***HTML:*** Structures the web pages
- ***CSS:*** Styles and enhances visual appearance
- ***JavaScript:*** Adds interactivity and dynamic behaviour

2. Responsive Design

The application interface is designed to be responsive, ensuring compatibility across different devices such as desktops, tablets, and mobile screens. This improves user experience and accessibility.

AI and Machine Learning Technologies

1. Groq API Integration

CodeSageAI uses Groq to perform intelligent code analysis. Groq provides high-speed AI inference capabilities, enabling fast and accurate responses.

The AI system is responsible for:

- Analyzing code structure and logic
- Detecting syntax and logical errors
- Suggesting improvements
- Providing explanations in a human-like format

2. Large Language Models (LLMs)

The system utilizes Large Language Models to understand programming logic and generate meaningful feedback. These models help in transforming raw code into structured insights, making the platform more interactive and educational.

Database: PostgreSQL

CodeSageAI uses PostgreSQL as its database system. It is a powerful, open-source relational database known for its reliability, scalability, and advanced features, making it suitable for both development and large-scale production environments.

The database is used to store:

- User data and authentication details
- Code submissions
- Analysis results

System Integration

The system integrates frontend, backend, and AI components into a unified platform. The workflow includes:

- User inputs or uploads code
- Backend processes the request
- AI analyzes the code using Groq API
- Results are returned and displayed in structured format

This integration ensures smooth communication between different components of the system.

Additional Libraries

Library	Version	Purpose
Pillow	12.2.0	Python Imaging Library for image processing, EXIF handling, and format conversion in the upload module
pytesseract	0.3.13	Python wrapper for Google's Tesseract OCR engine, used in the experimental text extraction feature
OpenCV (cv2)	4.13.0	Computer Vision library for advanced image pre-processing before OCR or AI analysis
python-dotenv	1.2.1	Environment variable management for securely loading API keys and configuration from .env files
gunicorn	25.1.0	Production-grade WSGI HTTP server for deploying the Django application
httpx	0.28.1	Modern HTTP client library used internally by the Groq SDK for API communication

Development Tools

Tool	Purpose
Visual Studio Code	Primary code editor with Python, Django, and Tailwind CSS extensions
Git & GitHub	Version control and collaborative code repository management
Postman	API endpoint testing and debugging for the Groq AI integration
Google Chrome DevTools	Frontend debugging, responsive design testing, and network request inspection
PostgreSQL	Visual database inspection and query testing during development

Comparison with Existing Systems

Feature	GitHub Copilot	ChatGPT	CodeGuru	CodeSageAI
AI Code Analysis	Partial	Yes	Yes	Yes (AI-based)
Bug Detection	Limited	Yes	Yes	Yes
Code Improvement Suggestions	Yes	Yes	Yes	Yes
Time & Space Complexity	No	Partial	No	Yes

Learning Support	Limited	Yes	Limited	Yes
Community Sharing	Yes (GitHub)	No	No	Yes
Focus Area	Code Completion	General AI Assistance	Code Review	AI Code Analysis & Learning

As shown in the above table, CodeSageAI provides several advanced features that are not available in traditional platforms. While existing systems such as GitHub Copilot, ChatGPT, and CodeGuru focus mainly on code assistance, content generation, or development support, they do not offer intelligent code analysis capabilities.

CodeSageAI stands out by integrating AI-powered features such as code analysis, bug detection, improvement suggestions, and performance evaluation. It also supports learning by providing structured explanations, making it highly useful for students and developers.

Therefore, CodeSageAI occupies a unique position by combining artificial intelligence with code learning and analysis, which is not offered by existing platforms.

REQUIREMENT ANALYSIS

Identification of Need

The need for CodeSageAI arises from the increasing complexity of software development and the growing demand for tools that can assist programmers in writing efficient, error-free, and optimized code. Despite the availability of compilers and development environments, many developers especially beginners face difficulties in understanding code logic, identifying errors, and improving performance. This creates a demand for an intelligent system that not only analyzes code but also provides meaningful explanations and suggestions for improvement.

1. Educational Need

Many students and beginner programmers struggle with learning programming concepts effectively. Traditional tools only indicate errors without explaining the underlying logic or providing guidance for improvement. This lack of structured learning support slows down the learning process and reduces confidence.

CodeSageAI addresses this need by providing:

- Step-by-step explanations of code
- Clear identification of mistakes
- Guidance on best coding practices

This helps users understand not just what is wrong, but also why it is wrong.

2. Technological Need

With the advancement of Artificial Intelligence and Large Language Models, there is an opportunity to enhance traditional coding tools with intelligent analysis capabilities. Existing tools such as GitHub Copilot and ChatGPT provide assistance, but they are not specifically designed for structured code evaluation and detailed feedback.

CodeSageAI fulfills this need by:

- Integrating AI for code analysis
- Providing real-time feedback
- Generating human-like explanations

3. Performance Optimization Need

Developers often focus on solving problems without considering efficiency. As a result, code may work correctly but perform poorly in terms of time and space complexity.

CodeSageAI addresses this by:

- Analyzing time complexity
- Evaluating space usage
- Suggesting optimized approaches

This encourages developers to write efficient and scalable code.

4. Practical Development Need

Debugging and code review are time-consuming tasks in software development. Developers need quick and reliable tools that can assist in identifying issues and improving code quality without manual effort.

CodeSageAI helps by:

- Automating bug detection
- Providing instant suggestions
- Reducing development time

5. Learning and Productivity Gap

There exists a gap between coding tools (which focus on execution) and learning platforms (which focus on theory). Developers need a system that combines both aspects.

CodeSageAI bridges this gap by:

- Combining analysis with explanation
- Supporting both learning and development
- Creating an interactive coding experience

Preliminary Investigation

A preliminary investigation was conducted to evaluate the feasibility, usefulness, and demand for the CodeSageAI system. The investigation focused on identifying current limitations in existing tools and understanding the requirements of users.

The following activities were carried out:

1. Study of Existing Systems

An analysis of existing platforms such as GitHub Copilot and ChatGPT was conducted to understand their capabilities and limitations.

Findings:

- These tools assist in code generation but lack structured code analysis
- Limited support for step-by-step explanations
- Do not always provide detailed performance analysis (time and space complexity)
- Not specifically designed as learning-oriented platforms

This analysis highlighted the need for a system focused on code understanding and improvement.

2. User Requirement Analysis

An informal study was conducted among students and beginner programmers to understand their challenges.

Observations:

- Many users struggle with debugging and understanding errors
- Difficulty in improving code efficiency
- Need for clear explanations and structured output
- Desire for a system that combines learning with coding assistance

Feasibility Study

1. Technical Feasibility

The project is technically feasible based on the following factors:

- The developer possesses the required skills in Python, Django, HTML, CSS, JavaScript, and AI/ML integration.
- The Groq API provides fast and reliable AI-based code analysis.
- The system architecture is simple and modular, making development and maintenance manageable.
- All required technologies and tools are stable, well-documented, and widely used.

2. Economic Feasibility

Item	Cost (INR)	Notes
Development Hardware	0	Existing laptop (Intel i5, 8GB RAM)
Django Framework	0	Open-source (BSD License)
Groq API	0	Free tier (14,400 tokens/min)
Cloudinary Storage	0	Free tier (25 credits/month)
Domain & Hosting (Optional)	~2,000/year	AWS Cloud (EC2)
Development Tools (VSCode, Git)	0	Open-source / free
Total Estimated Cost	~₹2,000	

3. Operational Feasibility

The system is designed to be user-friendly and accessible to both beginners and developers.

- Users can easily input or upload code for analysis
- The interface is simple and requires minimal technical knowledge
- The system provides clear, structured outputs for better understanding
- No specialized hardware or software is required

Thus, the system can be effectively used in real-world scenarios.

4. Time Feasibility

The project is planned within a defined academic timeline.

- Development follows a modular approach (authentication, code analysis, UI, etc.)
- Each module can be developed and tested independently
- The overall timeline is achievable within the semester duration

Functional Requirements

ID	Requirement	Priority
FR-01	System shall allow user registration and login	High
FR-02	System shall accept code input/upload from users	High
FR-03	System shall analyze code using AI	High
FR-04	System shall detect syntax and logical errors	High
FR-05	System shall provide code improvement suggestions	High
FR-06	System shall display time and space complexity	Medium
FR-07	System shall provide structured output format	High
FR-08	System shall support multiple programming languages	Medium
FR-09	System shall allow users to view analysis results	High
FR-10	System shall maintain history of analyzed code	Low

Non-Functional Requirements

ID	Requirement	Category
NFR-01	System response time should be fast (within a few seconds)	Performance
NFR-02	System should ensure secure handling of user data	Security
NFR-03	System should be user-friendly and easy to use	Usability
NFR-04	System should be scalable for future enhancements	Scalability
NFR-05	System should be reliable and available when needed	Reliability
NFR-06	System should support multiple devices (responsive UI)	Usability
NFR-07	System should maintain data consistency and accuracy	Integrity

PROBLEM DEFINITION

Problem Statement

With the rapid growth of software development, programmers especially beginners face several challenges in writing efficient, error-free, and optimized code. While many tools are available for code execution and generation, there is a lack of systems that provide structured analysis, clear explanations, and guidance for improvement. As a result, users often struggle to understand their mistakes, improve logic, and write high-quality code.

The key problems addressed by CodeSageAI are described below:

1. Lack of Code Understanding

Many programmers are able to write code but do not fully understand how it works. When errors occur, traditional compilers only display error messages without explaining the underlying logic.

- Users cannot easily identify why the error occurred
- Lack of step-by-step explanation reduces learning
- Difficulty in improving problem-solving skills

This creates a need for a system that explains code in a clear and structured manner.

2. Inefficient Debugging Process

Debugging is a time-consuming and complex process, especially for beginners.

- Errors are difficult to locate and fix
- Trial-and-error methods are commonly used
- Lack of intelligent guidance slows down development

There is a need for automated tools that can detect bugs and suggest fixes instantly.

3. Lack of Performance Awareness

Many developers focus only on correctness and ignore performance.

- No proper understanding of time complexity
- Inefficient use of memory (space complexity)
- Difficulty in optimizing code

This leads to solutions that work but are not efficient.

4. Limited Learning-Oriented Tools

Existing tools such as GitHub Copilot and ChatGPT assist in coding but are not specifically designed for structured learning.

- Focus more on code generation than explanation
- Lack of organized output format
- Not tailored for beginner-friendly learning

5. Gap Between Coding and Learning

There is a clear gap between development tools and educational platforms.

- Coding tools focus on execution
- Learning platforms focus on theory
- No integrated system that combines both

This results in inefficient learning and slower skill development.

Proposed Solution

CodeSageAI addresses the above problems by providing an AI-powered platform for intelligent code analysis and learning.

- ***AI-Based Code Analysis:*** The system analyzes user-submitted code to detect syntax errors, logical issues, and inefficiencies.
- ***Bug Detection and Suggestions:*** It automatically identifies errors and provides suggestions to fix them.
- ***Performance Evaluation:*** The system evaluates time and space complexity and suggests optimization techniques.
- ***Structured Explanations:*** CodeSageAI provides clear, step-by-step explanations to help users understand their code.
- ***Learning-Oriented Approach:*** The platform acts as a virtual coding assistant that improves both knowledge and productivity.

PLANNING & SCHEDULING

Project Planning

The CodeSageAI project follows a structured development approach to ensure timely completion and efficient resource utilization. The development process is divided into multiple phases including requirement analysis, design, implementation, testing, and deployment. The total project duration is approximately 16 weeks, aligned with the academic semester timeline. Each phase has clearly defined deliverables and milestones to track progress effectively.

Work Breakdown Structure (WBS)

Work Breakdown Structure (WBS) for CodeSageAI

	Activity	Duration	Deliverable
 Phase 1: Planning	1.1 Requirement Gathering	Week 1	SRS Document
	1.2 Feasibility Study	Week 1	Feasibility Report
	1.3 Technology Selection	Week 2	Tech Stack Document
 Phase 2: Design	2.1 System Architecture Design	Week 2-3	Architecture Diagram
	2.2 UI/UX Design	Week 3	Wireframes
	2.3 Database Design	Week 3-4	ER Diagram
 Phase 3: Development	3.1 User Authentication Module	Week 4-5	Login/Signup System
	3.2 File Upload System	Week 5-6	Upload Feature
	3.3 Code Analysis Engine (LLM Integration)	Week 6-8	AI Analysis Module
	3.4 Session Management	Week 7-8	History Tracking
	3.5 Frontend Integration	Week 8-9	UI Connected to Backend
 Phase 4: Testing	4.1 Unit Testing	Week 10	Test Cases
	4.2 Integration Testing	Week 10-11	Test Report
 Phase 5: Deployment	5.1 Deployment Setup	Week 12	Live System
	5.2 Bug Fixing & Optimization	Week 12-13	Optimized System
 Phase 6: Documentation	6.1 Report Writing	Week 14-15	Project Report
	6.2 Final Presentation	Week 15-16	PPT & Demo

The Work Breakdown Structure (WBS) of the CodeSageAI represents the systematic division of the project into smaller and manageable phases. It helps in organizing tasks, allocating resources, and tracking project progress efficiently throughout the development lifecycle.

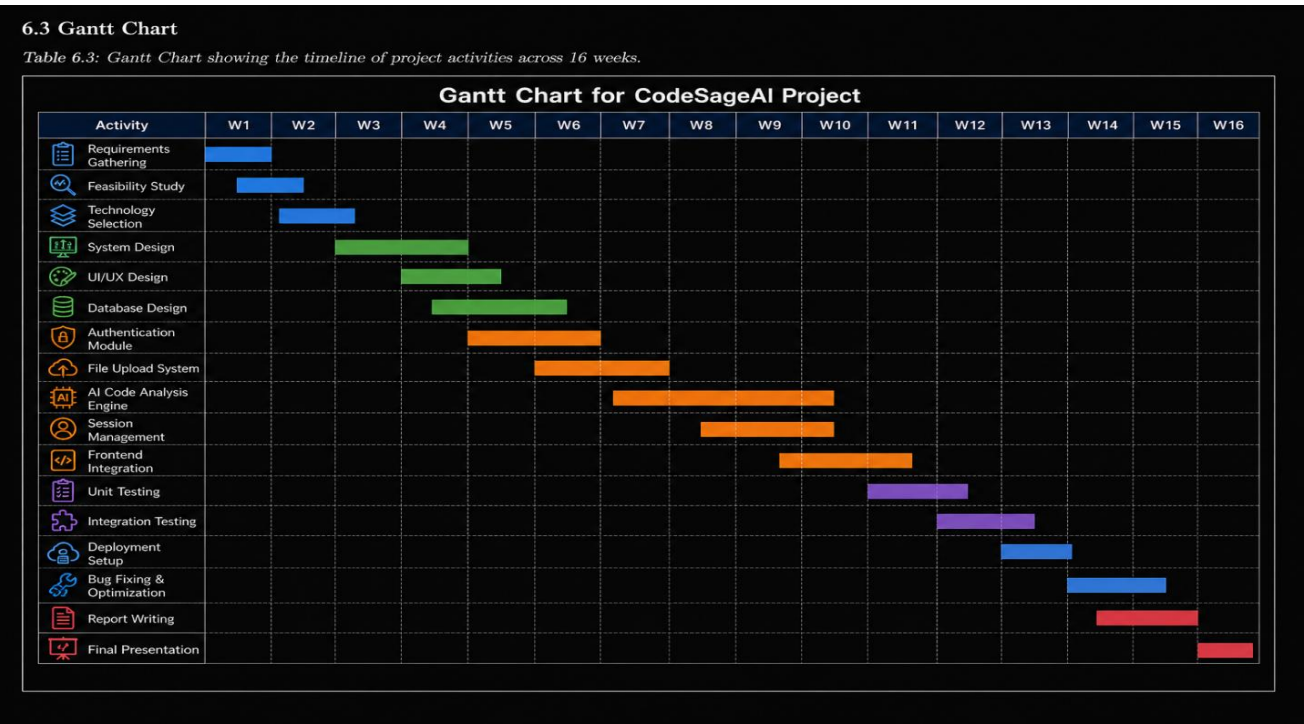
The project is divided into major phases including planning, design, development, testing, deployment, and documentation. Each phase contains specific activities and deliverables that contribute to the successful completion of the system. The WBS ensures proper scheduling and coordination among different project activities, allowing the development process to remain structured, time-efficient, and goal-oriented.

Gantt Chart

The Gantt Chart represents the timeline and scheduling of activities involved in the development of the CodeSageAI.

- Initial weeks focus on planning and requirement analysis
- Middle phase covers design and development modules
- Final phase includes testing, deployment, and documentation

It provides a visual overview of project phases, task durations, and the sequence of activities throughout the development lifecycle. The chart helps in tracking project progress, managing deadlines, and ensuring proper coordination between different development stages such as planning, design, implementation, testing, deployment, and documentation. It also helps identify overlapping tasks and improves overall project time management.

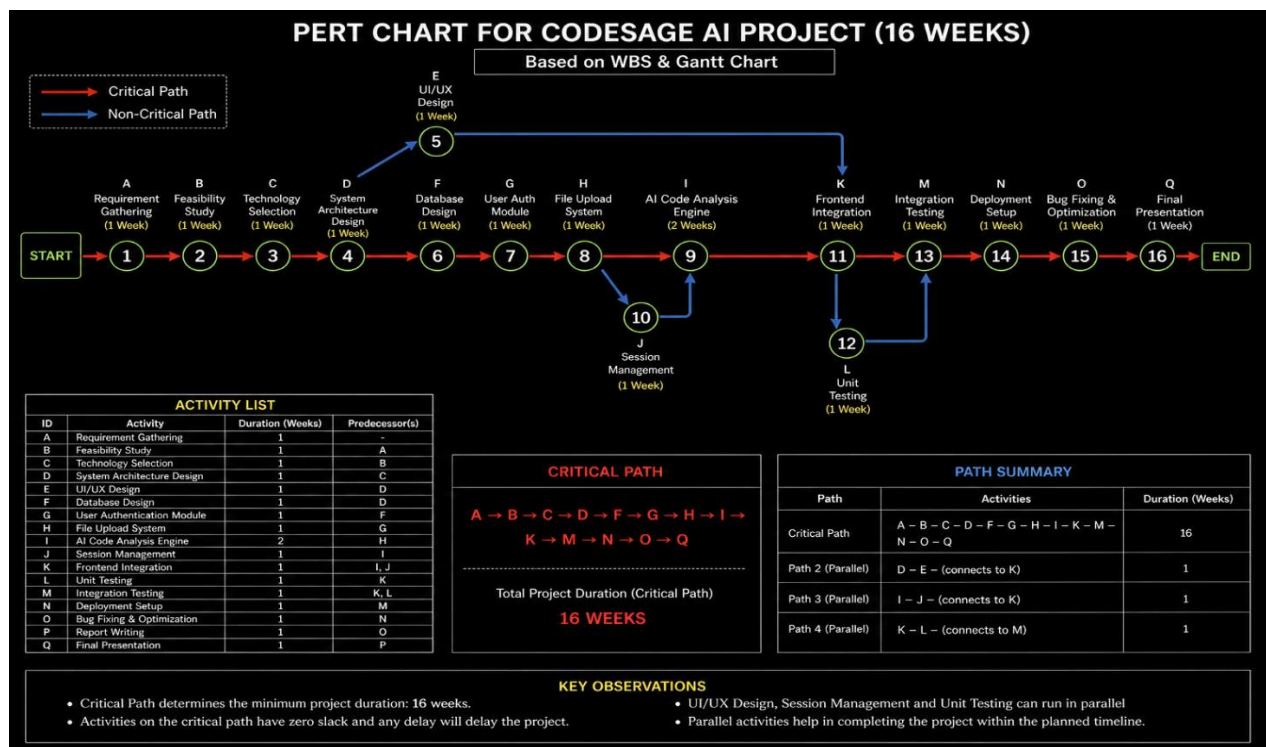


PERT Chart

The Program Evaluation and Review Technique (PERT) Chart is used to analyze and manage the sequence of activities involved in the development of the CodeSageAI. It helps in identifying task dependencies, estimating project duration, and determining the critical path for successful project completion.

The PERT chart provides a graphical representation of project scheduling by connecting various activities from planning to deployment. It assists in monitoring progress, improving time management, and ensuring that all project phases are completed within the planned timeline.

PERT Network Representation:



SOFTWARE REQUIREMENT SPECIFICATION (SRS)

Introduction

This Software Requirement Specification (SRS) document provides a detailed description of the CodeSageAI system, an AI-powered code analysis and learning platform. The purpose of this document is to define the system's functional and non-functional requirements, system features, constraints, and external interfaces. The document serves as a reference for developers, designers, and evaluators, ensuring that the system is developed according to specified requirements.

Overall Description

1. Product Perspective

CodeSageAI is a web-based application designed to assist users in understanding, analyzing, and improving programming code using artificial intelligence. The system integrates external AI services such as Groq API and LLM-based models to provide intelligent outputs. It operates as a standalone platform with a Django-based backend and a responsive frontend interface.

2. Product Functions

The system provides the following major functionalities:

- ***User Management:*** Registration, login, authentication, and profile handling
- ***Code Analysis:*** Understanding code structure and logic using AI models
- ***Code Explanation:*** Line-by-line and summary-based explanation generation
- ***Error Detection:*** Identification of syntax and logical errors
- ***Optimization Suggestions:*** Recommendations to improve code performance
- ***Chatbot Support:*** Interactive AI assistant for programming queries
- ***Search Functionality:*** Retrieval of past code analyses and responses
- ***Feedback System:*** User feedback collection for system improvement

3. User Classes and Characteristics

User Class	Description	Technical Proficiency
End User	Users who upload code for analysis and learning	Basic (students/beginners)
Developer	Users who explore advanced debugging and optimization features	Intermediate
Administrator	System manager who controls users, monitors activities, and maintains the system	Intermediate to Advanced

4. Operating Environment

Component	Specification
Operating System	Windows / Linux / macOS
Runtime	Python 3.10+
Web Framework	Django 5.2.11
Database	SQLite3 (development) / PostgreSQL (production-ready)
Web Server	Django Development Server (dev) / Gunicorn + Nginx (production)
Client Browser	Google Chrome 120+, Firefox 115+, Safari 17+, Edge 120+
External Services	Groq Cloud API, Cloudinary CDN

5. Design and Implementation Constraints

- The system depends on external AI APIs; downtime may affect functionality
- Code analysis accuracy depends on the AI model performance
- Internet connectivity is required for AI processing
- The system must ensure data security and user privacy
- File size and input limits are imposed for efficient processing

External Interface Requirements (Continued)

1. User Interface (Detailed Screens)

- **Authentication Pages:** Separate login and signup pages with proper form validation and error handling.
- **Dashboard (Home):** Main working interface where users upload code, view AI-generated

analysis, and interact with features. Displays code explanation, detected errors, and optimization suggestions in a structured format.

- **Chatbot Interface:** Interactive panel for asking programming-related queries and receiving AI responses.
- **Profile Page:** Allows users to view and update personal details and track activity history.

2. Hardware Interfaces

The system does not require any specialized hardware. It operates on standard computing devices:

- **Server:** Any machine capable of running Python 3.10+ with adequate memory and storage.
- **Client:** Desktop, laptop, tablet, or smartphone with a modern web browser.
- **Input Device:** Keyboard for code input and navigation.

3. Software Interfaces

Interface	Protocol	Data Format	Purpose
Groq Cloud API	HTTPS REST	JSON + Base64 images	AI-based code analysis and instruction generation
Django ORM	Python DB-API 2.0	SQL (abstracted)	Database CRUD operations
Browser Fetch API	HTTPS AJAX	JSON / FormData	Asynchronous client-server communication

4. Communication Interfaces

All communication between client and server occurs via HTTP/HTTPS protocols. The system uses Django's request-response model for handling user interactions.

- AJAX calls are used for asynchronous communication
- JSON is used as the primary data exchange format
- Secure headers and session handling ensure data protection

System Features — Detailed Specifications

1. User Registration (FR-01)

Description: New users can create an account by providing personal details such as name, email, and password.

Input: First name, last name, email, password (via HTML form submission)

Processing:

- System validates input fields
- Checks for duplicate email
- Password is securely hashed
- User account is created in the database

Output:

- Successful registration message and login access
- Error message in case of invalid input

2. Code Analysis (FR-02, FR-03, FR-04)

Description: The system analyzes user-submitted code and generates explanations, detects errors, and provides optimization suggestions.

Input: Source code file or text input

Processing:

- Code is parsed and formatted
- Sent to AI model via API
- AI generates structured response including explanation and insights

Output:

- Code summary
- Line-by-line explanation
- Error detection results
- Optimization suggestions

3. Chatbot Interaction (FR-05)

Description: Users can interact with an AI chatbot to ask programming-related questions.

Input: User queries in text format

Processing:

- Query is sent to AI model
- Context-based response is generated

Output:

- Real-time conversational response

4. Search Functionality (FR-06)

Description: Allows users to search previous code analyses and chatbot responses.

Input: Search keywords

Processing:

- Query processed against stored data
- Matching results retrieved

Output:

- List of relevant past records

5. Feedback System (FR-07)

Description: Users can provide feedback on system performance and AI outputs.

Input: Ratings, comments

Processing:

- Feedback stored in database
- Used for system improvement

Output:

- Confirmation of submission

SOFTWARE ENGINEERING PARADIGM APPLIED

Development Methodology: Agile–Waterfall Hybrid

The CodeSageAI project adopts a hybrid Agile–Waterfall development methodology that combines the structured planning of the Waterfall model with the flexibility and iterative nature of Agile practices. This approach is suitable for academic projects where proper documentation is required, while also allowing continuous improvements during development.

The Waterfall model is used in the initial phases such as requirement analysis, system design, and documentation, ensuring a well-defined structure. Agile principles are applied during development and testing phases, allowing iterative improvements, faster feedback, and flexibility in implementing features such as AI integration and user interface enhancements.

1. Waterfall Components

The following phases follow a sequential Waterfall approach:

- ***Requirement Analysis Phase:*** Complete gathering of system requirements and preparation of Software Requirement Specification (SRS).
- ***Design Phase:*** Creation of system architecture, database schema, UML diagrams, and UI wireframes before development begins.
- ***Documentation Phase:*** Preparation of project documentation, reports, and design records after implementation.

2. Agile Components

The following aspects follow Agile principles:

- ***Iterative Module Development:*** Each module (Code Analyzer, Explanation, Error Detection, Chatbot, etc.) is developed in small iterations with continuous refinement.
- ***Continuous Integration:*** Features are integrated into the main system as soon as they are developed and tested.
- ***Adaptability:*** The AI module and system features are improved continuously based on testing and feedback.
- ***Frequent Testing:*** Each module is tested individually during development to ensure early detection of errors.

Architectural Pattern: Model-View-Template (MVT)

Django follows the Model-View-Template (MVT) architectural pattern, which is Django's specific implementation of the broader Model-View-Controller (MVC) paradigm. In CodeSageAI, the MVT pattern is applied as follows:

MVT Component	MVC Equivalent	CodeSageAI Implementation
Model	Model	Defines user data, code submissions, analysis results, and feedback using Django models
View	Controller	Processes user input, calls AI APIs, and returns analysis results
Template	View	HTML templates render code input forms, results, chatbot UI, and dashboards

Design Principles Applied

The development of CodeSageAI follows key software engineering design principles to ensure maintainability, scalability, and performance:

- ***Separation of Concerns:*** Different functionalities such as UI, business logic, and data handling are separated into templates, views, and models respectively. \
- ***Modularity:*** The system is divided into independent modules (analysis, chatbot, authentication, etc.), making it easy to maintain and extend.
- ***DRY (Don't Repeat Yourself):*** Reusable components such as templates and functions are used to avoid code duplication.
- ***Security:*** Django's built-in features such as authentication, CSRF protection, and input validation are used to secure the system.
- ***Scalability:*** The system is designed to support additional features such as multi-language support and real-time analysis in the future.
- ***User-Centered Design:*** The interface is designed to be simple, intuitive, and beginner-friendly.

MODULE DESCRIPTIONS

Modules Developed by Pradyumna Singh (Roll No. 1230258295)

Module 1: Code Analyzer Module

Purpose: The Code Analyzer Module is responsible for processing the source code submitted by the user. It prepares the code for AI-based analysis by validating its structure and format.

Key Components:

- Input handling for code submission
- Programming language detection
- Syntax validation and formatting
- Preprocessing and structuring of code

Working: The user submits code through the interface, which is received by the backend. The module analyzes the structure, checks formatting consistency, and converts the input into a structured format suitable for AI processing. It ensures the code is clean and ready for further analysis.

Module 2: Explanation Module

Purpose: This module generates detailed explanations of the submitted code to help users understand program logic.

Key Components:

- Code summarization engine
- Line-by-line explanation generator
- Natural language formatting

Working: After AI processing, this module generates a high-level summary and detailed explanation of the code. The output is presented in simple language, enabling users to understand each part of the program clearly.

Module 3: Error Detection Module

Purpose: To identify and explain errors in the submitted code.

Key Components:

- Syntax error detection
- Logical error identification
- Error explanation system
- Correction suggestion generator

Working: The module analyzes the code for possible issues and highlights errors. It explains the cause of each error and suggests possible corrections, improving debugging efficiency.

Module 4: Optimization Module

Purpose: To improve code efficiency and performance.

Key Components:

- Time complexity analysis
- Space optimization suggestions
- Code readability improvements

Working: This module evaluates the performance of the code and suggests optimized solutions. It also recommends best practices for clean and maintainable coding.

Module 5: Chatbot Module

Purpose: To provide an interactive AI assistant for programming-related queries.

Key Components:

- LLM-based conversational engine
- Query-response handler
- Context management

Working: Users interact with the chatbot by asking questions. The module processes queries and generates real-time responses, supporting continuous learning and doubt resolution.

Modules Developed by Vishal Chaudhary (Roll No. 1230258472)

Module 6: Account Module

Purpose: To manage user authentication and system access.

Key Components:

- User registration and login
- Password management
- Session handling

Working: This module handles authentication processes using backend logic. It ensures secure login and maintains user sessions throughout system usage.

Module 7: Admin Module

Purpose: To provide administrative control over the system.

Key Components:

- User management
- Activity monitoring
- Database management

Working: The admin accesses the system through a control panel to manage users, monitor system activity, and maintain database operations.

Module 8: Profile Module

Purpose: To manage user profile information.

Key Components:

- Profile data storage
- Image upload
- Activity tracking

Working: Users can view and update their personal information. The module stores user data and tracks system usage.

Module 9: Search Module

Purpose: To retrieve stored data efficiently.

Key Components:

- Keyword search
- Filtering and sorting
- Data retrieval system

Working: Users can search for previous code submissions and results. The module processes queries and returns relevant data quickly.

Module 10: Feedback Module

Purpose: To collect user feedback for system improvement.

Key Components:

- Feedback submission system
- Rating mechanism
- Data storage and analysis

Working: Users provide feedback on system performance and AI responses. The data is stored and used to improve system accuracy and user experience.

LOGICAL DESIGN (DFDs)

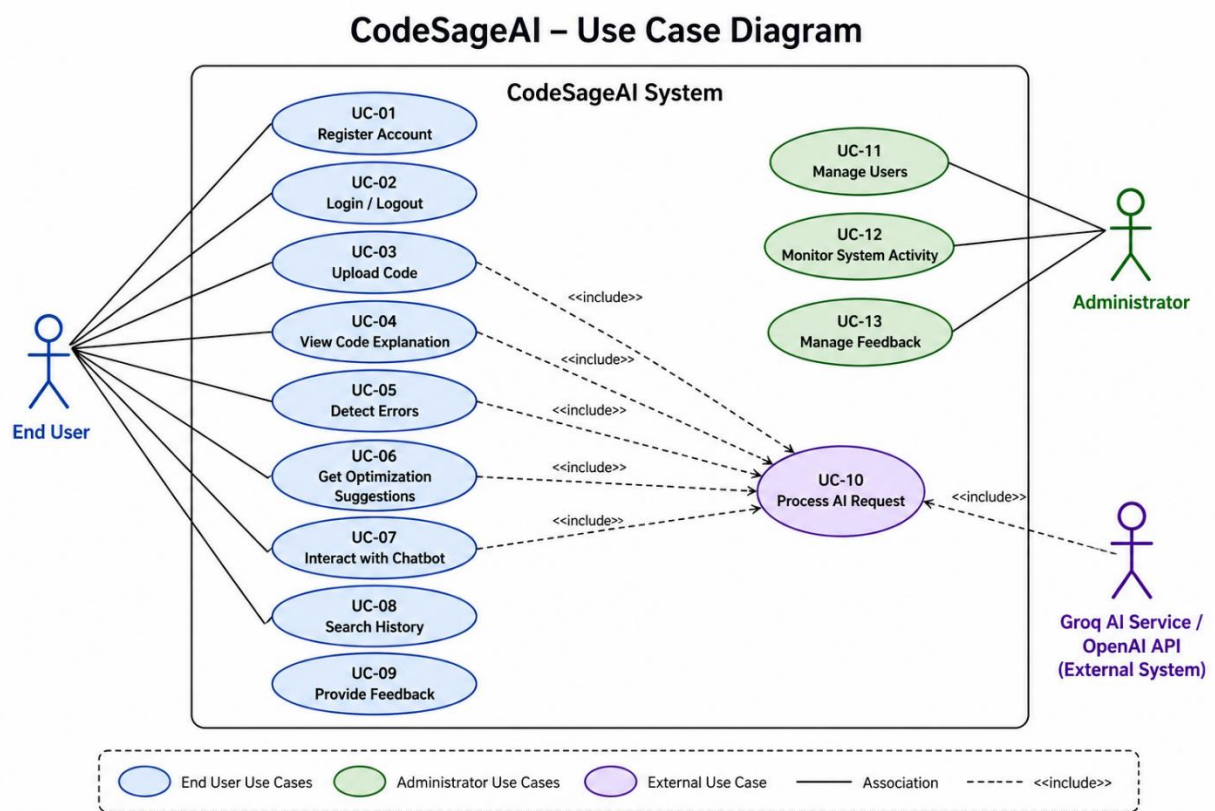
Use Case Diagram

The Use Case Diagram represents the interaction between different users and the CodeSageAI system. It identifies the primary actors and their associated functionalities within the system. The two main actors in the system are:

End User: Interacts with the system for code analysis and learning

Administrator: Manages system operations and user activities

The diagram illustrates how users interact with various modules such as authentication, code analysis, chatbot, and management features.

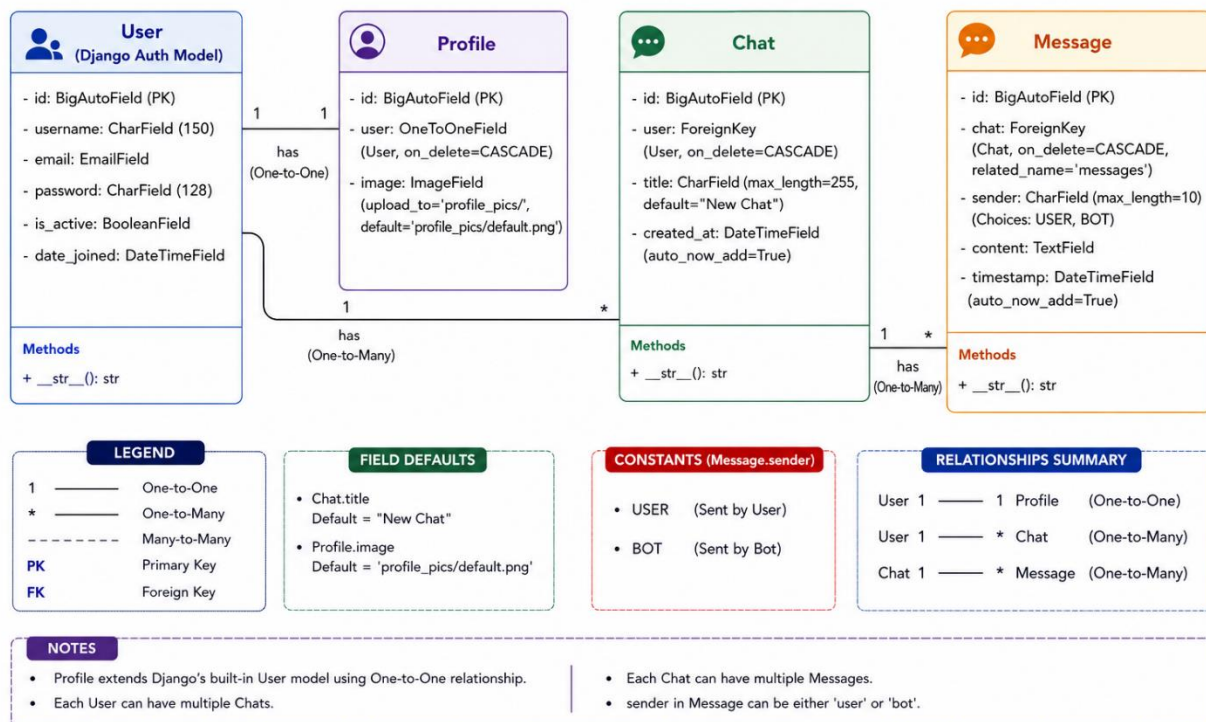


Class Diagram

The Class Diagram represents the structural design of the CodeSageAI system by illustrating the Django model classes, their attributes, methods, and relationships. It provides a clear view of how different classes interact with each other within the application.

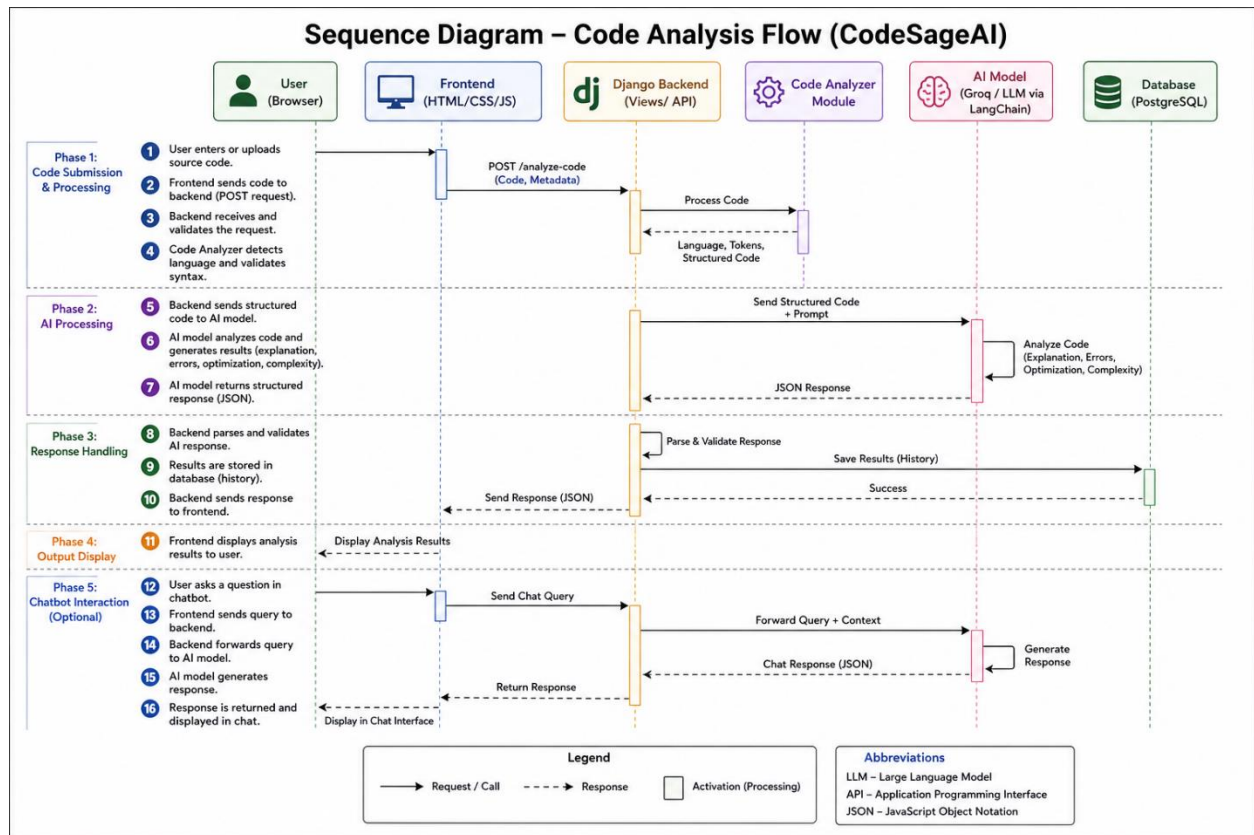
The diagram includes important entities such as User, Profile, Chat, and Message, along with their associations and data relationships. It helps in understanding the object-oriented structure of the system and supports efficient database and application design.

CodeSage AI – Django Models Class Diagram



Sequence Diagram — Code Analysis Flow (CodeSageAI)

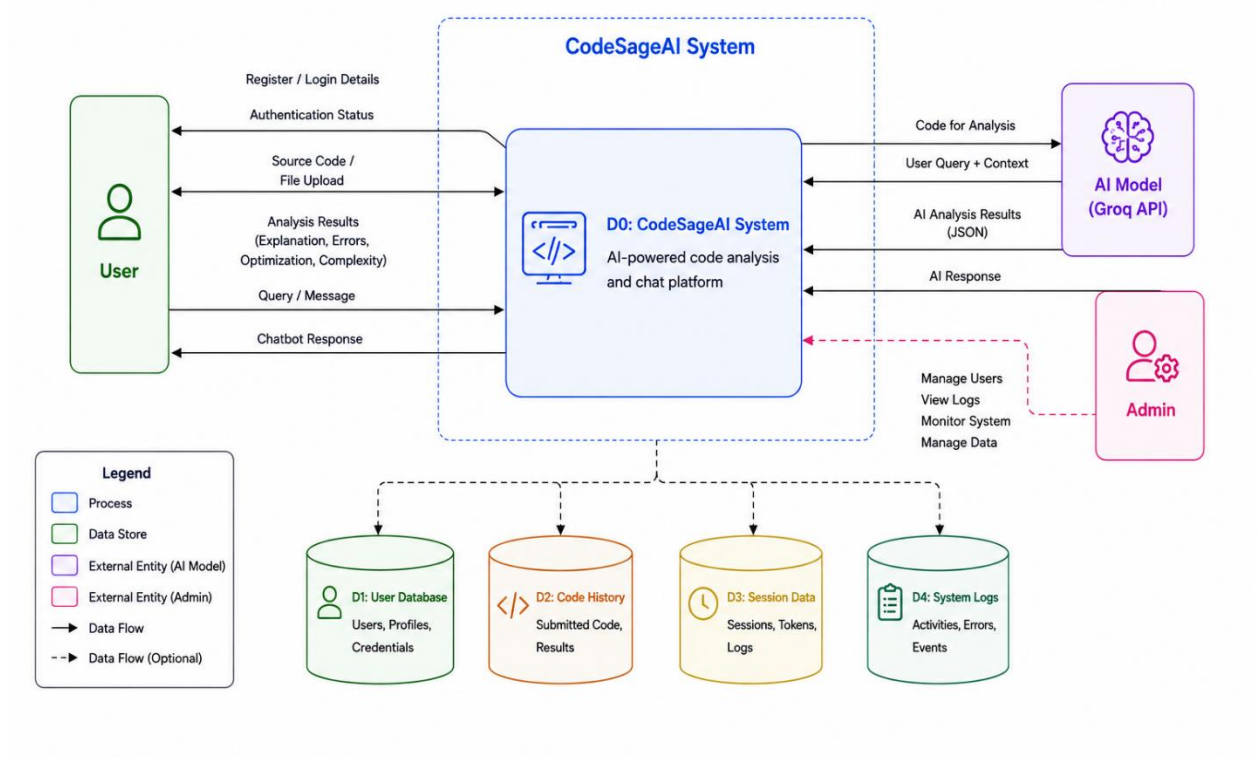
The sequence diagram represents how CodeSageAI processes user-submitted code through multiple modules such as analysis, explanation, error detection, optimization, and chatbot interaction using an AI model. It shows the interaction between the user, frontend, backend, and AI services.



Data Flow Diagram (Level 0)

The Level-0 Data Flow Diagram (DFD) provides a high-level overview of how data flows through the CodeSageAI system. It represents the interaction between external entities, the system, and major processes involved in code analysis and AI-based assistance. At this level, the system is shown as a single process with key data inputs and outputs.

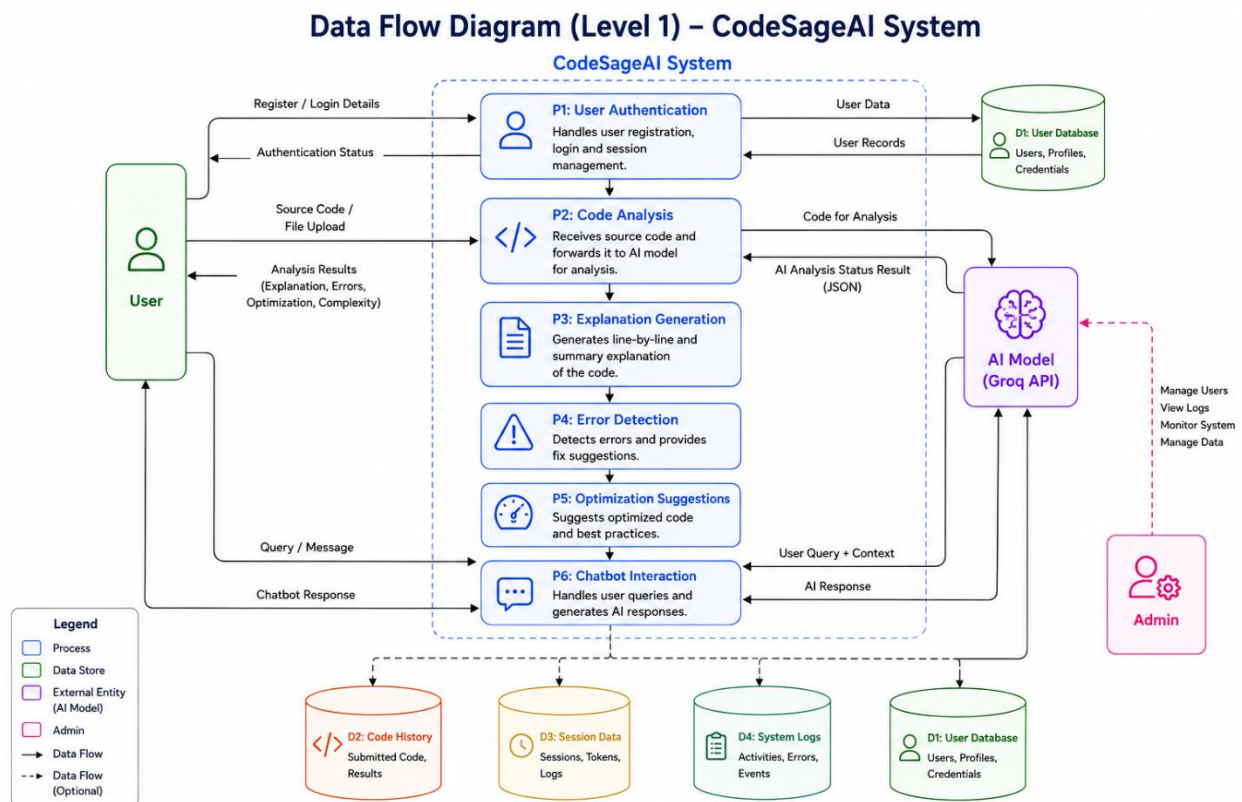
Data Flow Diagram (Level 0) – CodeSageAI System



Data Flow Diagram (Level 1)

The Level-1 Data Flow Diagram (DFD) provides a detailed view of the internal processes of the CodeSageAI system. It breaks down the main system into multiple sub-processes such as user authentication, code analysis, explanation generation, error detection, optimization suggestions, and chatbot interaction.

At this level, the flow of data between users, the AI model, system processes, and data stores is clearly represented to show how the system processes user requests and generates AI-powered responses.

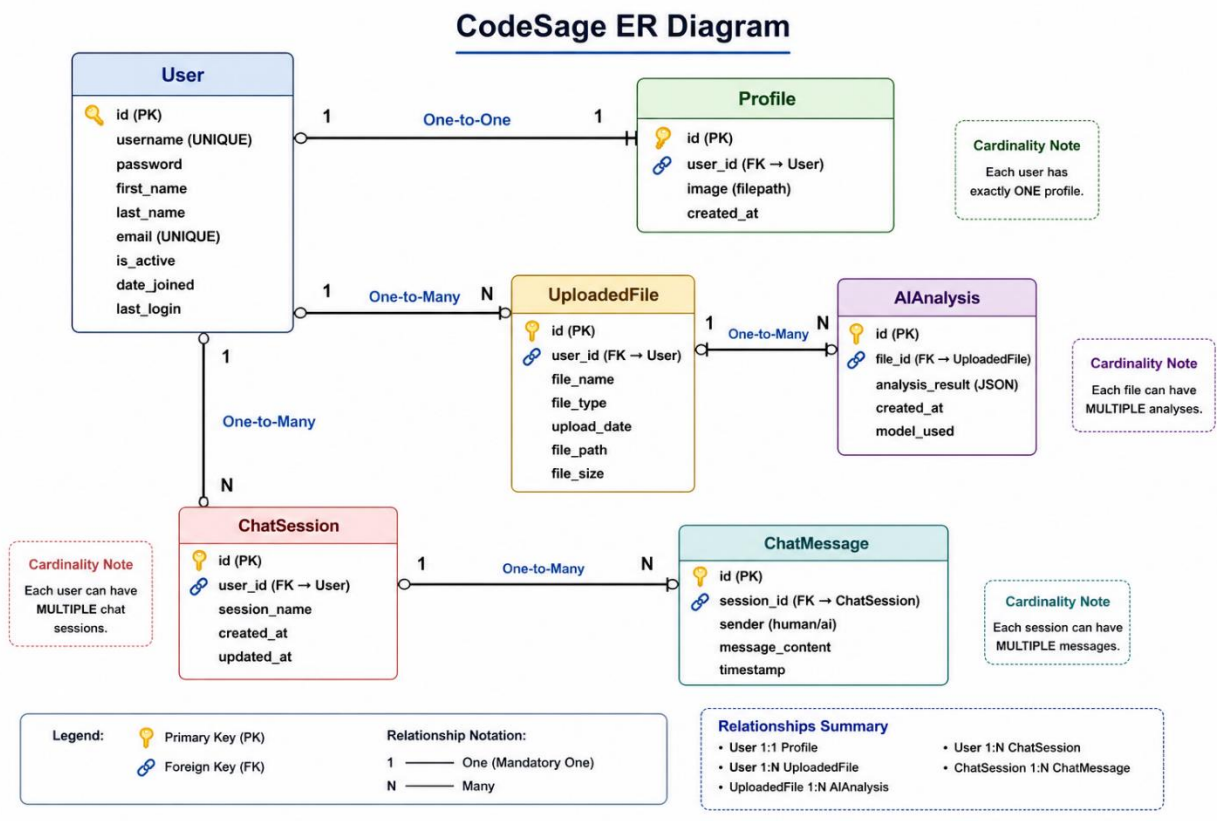


DATABASE DESIGN

Entity–Relationship (ER) Diagram

The Entity–Relationship (ER) diagram represents the logical structure of the database used in the system. It defines the various entities involved in the application, their attributes, and the relationships between them. The ER model provides a clear visualization of how data is stored, organized, and interconnected, ensuring efficient data management and consistency.

In this system, the database is designed using a relational model implemented through PostgreSQL. The schema supports multiple modules such as user management, code analysis, interaction history, and administrative control.



USER INTERFACE DESIGN

Design Philosophy

The User Interface (UI) of CodeSageAI is designed using a modern dark-themed layout inspired by a futuristic coding environment. The interface emphasizes simplicity, clarity, and smooth user interaction while maintaining a visually appealing aesthetic.

The design follows these core principles:

- **Modern Dark Theme:** The application uses a dark background with neon green highlights, creating a coding-centric environment that reduces eye strain and enhances focus.
- **Minimalistic Design:** Clean layouts with centered components and limited distractions ensure that users can focus on core functionalities like chatting and analysis.
- **Consistency:** Uniform color schemes, rounded input fields, and consistent button styles are used across all pages such as Login, Signup, Dashboard, and Settings.
- **User Feedback:** Buttons, hover effects, and form interactions provide immediate visual feedback, improving usability.
- **Accessibility & Simplicity:** The interface is intuitive and easy to navigate, making it suitable for both beginners and experienced users.

Page-by-Page UI Description

1. Login Page

The login page provides secure access to the system.

Key UI elements include:

- Centered login card with glowing border effect
- Input fields for username and password
- Login button with highlighted color
- Links for Forgot Password and Sign Up
- Matrix-style animated background enhancing visual appeal

2. Registration Page

The signup page allows new users to create an account.

Features include:

- Input fields for username, first name, last name, email, password, and confirm password
- Clean form layout with proper spacing

- “Sign Up” button for account creation
- Link to redirect existing users to login page

3. Dashboard (Main Interface)

The dashboard acts as the central workspace of the application.

Key components include:

Sidebar Navigation:

- New Chat
- History

Top Bar:

- Username display

Main Chat Area:

- Large workspace for conversation

Input Section:

- Text input field for queries
- “Send” button
- Add () button for additional actions

This layout ensures smooth interaction with the AI assistant.

4. Profile Page

The profile page displays user information.

Features include:

- Profile avatar with glowing circular border
- Option to remove or update profile image

Display of:

- Username
- Full Name
- Email

5. About Page

The About page provides information about the system.

Sections include:

- Application logo and title
- Introduction describing AI capabilities

- Mission statement explaining project goals

6. Settings Page

This page allows users to manage account settings.

Features include:

Editable fields:

- Username
- Email
- First Name
- Last Name
- “Save Changes” button

Danger Zone section:

- Option to delete account
- Confirmation input required before deletion

7. Contact Page

The contact page allows communication with the developer.

Includes:

Developer details:

- Name
- Phone number
- Email
- GitHub and LinkedIn links

Contact form with:

- Name
- Email
- Message field
- “Send Message” button

8. Help & Support Page

This section assists users in understanding the system.

Includes:

- Instructions on how to use the platform
- Frequently Asked Questions (FAQs)

- Tips for better results

Interface Workflow

The application follows a simple workflow:

- User logs in or registers
- Accesses the dashboard
- Starts a new chat or views history
- Enters a query
- Receives AI-generated response
- Manages profile and settings if needed

Responsive Design

All pages of CodeSageAI are designed to be fully responsive using modern CSS techniques, ensuring compatibility across different screen sizes and devices. The interface adapts dynamically to provide an optimal user experience on mobile phones, tablets, and desktops.

The responsive design is implemented using flexible layouts, grid systems, and media queries.

Responsive Features:

- Fluid layouts that automatically adjust based on screen size
- Flexible input fields and buttons for better accessibility
- Adaptive navigation (sidebar and top bar adjustments)
- Scalable typography for readability across devices
- Optimized spacing and alignment for different resolutions

Breakpoint	Width	Adaptation
Mobile (default)	< 768px	Single column layout, stacked components, full-width input fields
Tablet (md)	768px – 1024px	Two-column layout, adjusted spacing, compact sidebar
Desktop (lg)	> 1024px	Full layout with sidebar navigation, large chat area, multi-section display

CODING

This chapter presents the source code for the key files of the CodeSageAI platform. The code is organized by component: project configuration, data models, business logic, AI integration, admin panel, URL routing, and frontend templates.

settings.py — Django Project Configuration

File:core/settings.py/Lines:127

```
"""
```

```
Django settings for CSAI project.
```

```
Generated by 'django-admin startproject' using Django 6.0.3.
```

```
For more information on this file, see
```

```
https://docs.djangoproject.com/en/6.0/topics/settings/
```

```
For the full list of settings and their values, see
```

```
https://docs.djangoproject.com/en/6.0/ref/settings/
```

```
"""
```

```
from pathlib import Path
```

```
import os
```

```
# Build paths inside the project like this: BASE_DIR / 'subdir'.
```

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

```
# Quick-start development settings - unsuitable for production
```

```
# See https://docs.djangoproject.com/en/6.0/howto/deployment/checklist/
```

```
# SECURITY WARNING: keep the secret key used in production secret!
```

```
SECRET_KEY = 'django-insecure-s-xq%_-%#1jwy3=47@#^&b-f8(l'
```

```
# SECURITY WARNING: don't run with debug turned on in production!
```

```
DEBUG = True
```

```
ALLOWED_HOSTS = []
```

```
# Application definition
```

```
INSTALLED_APPS = [
```

```
    'django.contrib.admin',
```

```
    'django.contrib.auth',
```

```
    'django.contrib.contenttypes',
```

```
    'django.contrib.sessions',
```

```
    'django.contrib.messages',
```

```
    'django.contrib.staticfiles',
```

```

    'home',
]
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
ROOT_URLCONF = 'CSAI.urls'
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [os.path.join(BASE_DIR, 'templates')],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
WSGI_APPLICATION = 'CSAI.wsgi.application'

# Database
# https://docs.djangoproject.com/en/6.0/ref/settings/#databases
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'codesageAi',
        'USER': 'postgres',
        'PASSWORD': 'xxxxxxxxxx',
        'HOST': 'localhost',
    }
}

```

```

        'PORT': '5432',
    }
}

# Password validation
# https://docs.djangoproject.com/en/6.0/ref/settings/#auth-password-validators
AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]

# Internationalization
# https://docs.djangoproject.com/en/6.0/topics/i18n/
LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True

# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/6.0/howto/static-files/
STATIC_URL = 'statics/'
STATIC_URL = 'statics/'
STATICFILES_DIRS=[os.path.join(BASE_DIR,"statics")]
STATIC_ROOT=os.path.join(BASE_DIR,"assets")
MEDIA_URL = '/media/'
MEDIA_ROOT=os.path.join(BASE_DIR,"media")

```

models.py — Data Models

File:app/models.py/Lines:35

```
from django.db import models
from django.contrib.auth.models import User
# Create your models here.
class Profile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    image = models.ImageField(
        upload_to='profile_pics/',
        default='profile_pics/default.png'
    )
    def __str__(self):
        return self.user.username
class Chat(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    title = models.CharField(max_length=255, default="New Chat")
    created_at = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return self.title
class Message(models.Model):
    chat = models.ForeignKey(Chat, on_delete=models.CASCADE, related_name='messages')
    sender = models.CharField(max_length=10) # user / bot
    content = models.TextField()
    timestamp = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return self.sender
```

views.py — Business Logic (View Functions)

File:app/views.py/Lines: 318

"""

views.py

Clean version:

- Authentication
- Dashboard (Chat system)
- Profile & Settings

- File upload (AI analysis)

```
"""
# =====
# IMPORTS
# =====
from django.shortcuts import render, redirect, get_object_or_404
from django.http import JsonResponse
from django.contrib.auth.models import User
from django.contrib import messages
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.http import JsonResponse
from .models import Chat, Message, Profile
from .register import validate_signup
from home.upload import analyze_with_langchain, is_code
# =====
# 🗝️ AUTHENTICATION
# =====
def login_view(request):
    if request.method == "POST":
        user = authenticate(
            request,
            username=request.POST.get('username'),
            password=request.POST.get('password')
        )

        if user:
            login(request, user)
            return redirect('dashboard')

        messages.error(request, "Invalid username or password")

    return render(request, 'login.html')

def signup(request):
```

```

if request.method == "POST":
    username = request.POST.get('username')
    first_name = request.POST.get('firstname')
    last_name = request.POST.get('lastname')
    email = request.POST.get('email')
    password = request.POST.get('password')
    confirm_password = request.POST.get('confirm_password')

    error = validate_signup(username, email, password, confirm_password)

    if error:
        messages.error(request, error)
        return redirect('signup')

    User.objects.create_user(
        username=username,
        first_name=first_name,
        last_name=last_name,
        email=email,
        password=password
    )

    messages.success(request, "Account created successfully")
    return redirect('login')

return render(request, 'signup.html')

def logout_view(request):
    logout(request)
    return redirect('login')

# =====
#  DASHBOARD (CHAT SYSTEM)
# =====
@login_required
def dashboard(request, chat_id=None):

```

```

chats = Chat.objects.filter(user=request.user).order_by('-created_at')

current_chat = None
messages_list = []

if chat_id:
    current_chat = get_object_or_404(Chat, id=chat_id, user=request.user)

if request.method == "POST":
    content = request.POST.get("message")

    if content:
        # Load last 9 messages as per-chat context BEFORE saving the new one
        history_qs = current_chat.messages.order_by('-timestamp')[:9]
        chat_history = [
            {"role": m.sender, "content": m.content}
            for m in reversed(list(history_qs))
        ]

        # Save user message
        user_msg = Message.objects.create(
            chat=current_chat,
            sender='user',
            content=content
        )

        # Auto-title on first message
        if current_chat.title == "New Chat":
            clean_title = content.strip().split('\n')[0][:40]
            current_chat.title = clean_title if clean_title else "New Chat"
            current_chat.save()

        # Detect file extension hint from content if code block present
        ext = "txt"

        if content.startswith("```") and "\n" in content:
            first_line = content.split("\n")[0].strip("```").strip()

```



```

        if first_line:
            ext = first_line
        # Call AI with per-chat history (no global session leakage)
        reply = analyze_with_langchain(content, ext, chat_history)

        ai_msg = Message.objects.create(
            chat=current_chat,
            sender='ai',
            content=reply
        )

        return JsonResponse({
            "user": user_msg.content,
            "ai": ai_msg.content
        })

    messages_list = current_chat.messages.all()

    return render(request, 'dashboard.html', {
        'chats': chats,
        'messages': messages_list,
        'current_chat': current_chat
    })

# =====
#  NEW CHAT
# =====
@login_required
def new_chat(request):
    chat = Chat.objects.create(user=request.user)
    return redirect('dashboard_chat', chat_id=chat.id)

# =====
#  CLEAR CHAT
# =====
@login_required
def clear_chat(request, chat_id):

```

```

chat = get_object_or_404(Chat, id=chat_id, user=request.user)
chat.messages.all().delete()
return redirect('dashboard_chat', chat_id=chat.id)
# =====

# 👤 PROFILE
# =====

@login_required
def profile(request):
    profile, _ = Profile.objects.get_or_create(user=request.user)

    if request.method == 'POST':

        # Remove profile image
        if 'remove_photo' in request.POST:
            if profile.image and profile.image.name != 'profile_pics/default.png':
                profile.image.delete(save=False)

            profile.image = 'profile_pics/default.png'
            profile.save()
            return redirect('profile')

        # Upload new image
        if request.FILES.get('image'):
            if profile.image and profile.image.name != 'profile_pics/default.png':
                profile.image.delete(save=False)

            profile.image = request.FILES['image']
            profile.save()
            return redirect('profile')

    return render(request, 'profile.html', {'profile': profile})
# =====

# ⚙️ SETTINGS
# =====

@login_required

```

```

def setting(request):
    user = request.user
    if request.method == "POST":

        # Delete account
        if 'delete_account' in request.POST:
            confirm = request.POST.get('confirm_text')

            if confirm != "DELETE":
                messages.error(request, "Type DELETE to confirm account deletion")
                return redirect('settings')

            user.delete()
            messages.success(request, "Account deleted successfully")
            return redirect('login')

        # Update profile
        username = request.POST.get('username')
        email = request.POST.get('email')
        first_name = request.POST.get('first_name')
        last_name = request.POST.get('last_name')

        # Check duplicates
        if User.objects.exclude(id=user.id).filter(username=username).exists():
            messages.error(request, "Username already taken")
            return redirect('settings')

        if User.objects.exclude(id=user.id).filter(email=email).exists():
            messages.error(request, "Email already used")
            return redirect('settings')

        user.username = username
        user.email = email
        user.first_name = first_name
        user.last_name = last_name
        user.save()

```

```

        messages.success(request, "Profile updated successfully")
        return redirect('settings')

    return render(request, 'settings.html')
# =====

# 📁 FILE UPLOAD (AI ANALYSIS)
# =====

@login_required
def upload_file(request):
    if request.method != "POST":
        return JsonResponse({"error": "Invalid request"}, status=400)

    file = request.FILES.get("file")

    if not file:
        return JsonResponse({"error": "No file uploaded"}, status=400)

    try:
        code = file.read().decode("utf-8")
        ext = file.name.split('.')[-1]

        # File uploads have no prior context — analyzed as standalone
        result = analyze_with_langchain(code, ext, chat_history=None)

        return JsonResponse({"result": result})

    except Exception as e:
        return JsonResponse({"error": str(e)}, status=500)
# =====

# 📄 STATIC PAGES
# =====

@login_required
def about(request):
    return render(request, 'about.html')

```

```

@login_required
def helps(request):
    return render(request, 'help.html')

@login_required
def contact(request):
    return render(request, 'contact.html')

@login_required
def delete_chat(request, chat_id):
    if request.method == "POST":
        chat = get_object_or_404(Chat, id=chat_id, user=request.user)
        chat.delete()
        return JsonResponse({"success": True})
# =====
#  GET MESSAGES (API)
# =====
@login_required
def get_messages(request, chat_id):
    """Return all messages for a chat as JSON (used by dashboard.js)."""
    chat = get_object_or_404(Chat, id=chat_id, user=request.user)
    msgs = chat.messages.order_by('timestamp').values('sender', 'content', 'timestamp')
    return JsonResponse({
        "messages": [
            {"role": m["sender"], "content": m["content"]}
            for m in msgs
        ]
    })

```

upload.py — AI API Integration

File:app/upload.py/ Lines:173

"""

upload.py

AI Chat & Code Audit Service using LangChain + Groq.

Modes:

- Normal chat → friendly AI assistant
- Code input → structured code quality audit with suggestions

"""

```
import os
```

```
import re
```

```
from dotenv import load_dotenv
```

```
from langchain_groq import ChatGroq
```

```
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage
```

```
# ===== ENV SETUP =====
```

```
load_dotenv()
```

```
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
```

```
# ===== CODE DETECTION =====
```

```
_CODE_KEYWORDS = [
```

```
    "def ", "class ", "import ", "from ", "#include", "function ", "const ",
```

```
    "let ", "var ", "=>", "return ", "if (", "for (", "while (", "async ",
```

```
    "public ", "private ", "protected ", "void ", "int ", "string ", "bool ",
```

```
    "elif ", "except ", "lambda ", "SELECT ", "INSERT ", "UPDATE ", "DELETE ",
```

```
    "CREATE TABLE", "ALTER TABLE",
```

```
]
```

```
_CODE_EXTS = {
```

```
    "py", "js", "ts", "jsx", "tsx", "java", "c", "cpp", "cs", "go",
```

```
    "rb", "php", "swift", "kt", "rs", "html", "css", "sql", "sh",
```

```
    "yaml", "yml", "json", "xml",
```

```
}
```

```
def is_code(text: str, ext: str = "") -> bool:
```

```
    """Detect whether input is code rather than natural language."""
```

```
    if ext and ext.lower() in _CODE_EXTS:
```

```
        return True
```

```

# Explicit fenced code block
if "```" in text:
    return True

lines = text.strip().split("\n")
keyword_hits = sum(1 for k in _CODE_KEYWORDS if k in text)
symbol_count = len(re.findall(r"[{};]", text))
indented_lines = sum(1 for l in lines if l.startswith(("  ", "\t")))

if keyword_hits >= 2:
    return True
if symbol_count >= 3 and len(lines) >= 3:
    return True
if indented_lines >= 3 and len(lines) >= 5:
    return True

return False

# ===== SYSTEM PROMPTS =====
_CHAT_SYSTEM = """You are CodeSageAI, a friendly and knowledgeable AI assistant.

Guidelines:
- Answer conversationally and naturally.
- Use markdown (bold, lists, code snippets) when it adds clarity.
- For coding questions without a code snippet, give guidance and short examples.
- Keep replies focused — don't pad with filler.
- Be supportive and encouraging."""

_AUDIT_SYSTEM = """You are CodeSageAI, an expert code reviewer and software quality
engineer.

```

When given code, produce a structured audit using EXACTLY this format:

```
---
```

```
## Code Overview
```

One-paragraph summary of what the code does and its purpose.

Code Quality ☆☆☆☆☆ (rate 1–5)

- Readability & naming conventions
- Structure and organisation
- Adherence to language best practices

Issues Found

List each issue with a severity tag:

● ****Critical**** — bugs, crashes, data loss risk

□ ****Warning**** — logic errors, bad patterns

● ****Info**** — style, minor improvements

If no issues: "☑ No significant issues found."

Security Concerns

Any vulnerabilities (injection, hardcoded secrets, unsafe inputs, etc.).

If none: "☑ No security concerns identified."

Performance

Inefficiencies, unnecessary loops, memory issues, or O(n) surprises.

If none: "☑ Performance looks good."

Suggestions for Improvement

Numbered, actionable list. Include brief code snippets for clarity.

Refactored Snippet (if applicable)

Show a cleaner version of the most problematic section.

Be specific, educational, and actionable. Mention the language/framework when relevant. """

```
# ===== MAIN FUNCTION =====
```

```
def analyze_with_langchain(content: str, ext: str = "txt", chat_history: list = None) -> str:
```

```
    """
```

```
        Process user input and return an AI response.
```


Args:

content: The user's message or code.

ext: File extension hint (e.g. 'py', 'js', 'txt').

chat_history: Per-chat message history as a list of
{"role": "user"/"ai", "content": str} dicts.

Used to maintain conversation context per chat session.

Returns:

AI response string.

"""

if not GROQ_API_KEY:

return "✗ API key not configured. Set GROQ_API_KEY in your .env file."

if not content.strip():

return "✗ No input provided."

try:

llm = ChatGroq(
 model="llama-3.3-70b-versatile",
 temperature=0.7,
 api_key=GROQ_API_KEY,
)

code_mode = is_code(content, ext)

system_prompt = _AUDIT_SYSTEM if code_mode else _CHAT_SYSTEM

lc_messages = [SystemMessage(content=system_prompt)]

Inject per-chat conversation history for context

for msg in (chat_history or []):

role = msg.get("role", "")

text = msg.get("content", "")

if role == "user":

lc_messages.append(HumanMessage(content=text))

elif role in ("ai", "assistant"):

```

        lc_messages.append(AIMessage(content=text))
# Build the current user message
if code_mode:
    lang = ext if ext and ext not in ("txt", "md", "") else ""
    fence = f"```${lang}" if lang else "```"
    user_message = f"Please audit this code:\n\n{fence}\n{content}\n\n"
else:
    user_message = content

lc_messages.append(HumanMessage(content=user_message))

response = llm.invoke(lc_messages)
return response.content or "⚠️ No response generated."

except Exception as e:
    return f"❌ Error: {str(e)}"

```

admin.py — Django Admin Configuration

File:app/admin.py/Lines: 6

```

from django.contrib import admin
from home.models import Profile

```

Register your models here.

```

admin.site.register(Profile)

```

urls.py — URL Routing Configuration

File:app/urls.py/Lines:58

```

"""

```

URL Configuration for CodeSageAI app

Clean version:

- Authentication
- Dashboard (Chat system)
- Profile & static pages

- File upload

"""

```
from django.urls import path
```

```
from django.conf import settings
```

```
from django.conf.urls.static import static
```

```
from . import views
```

```
urlpatterns = [
```

```
    # ===== AUTHENTICATION =====
```

```
    path("", views.login_view, name='login'),
```

```
    path('signup/', views.signup, name='signup'),
```

```
    path('logout/', views.logout_view, name='logout'),
```

```
    # ===== DASHBOARD (CHAT SYSTEM) =====
```

```
    path('dashboard/', views.dashboard, name='dashboard'),
```

```
    path('dashboard/<int:chat_id>/', views.dashboard, name='dashboard_chat'),
```

```
    path('new-chat/', views.new_chat, name='new_chat'),
```

```
    path('clear-chat/<int:chat_id>/', views.clear_chat, name='clear_chat'),
```

```
    # ===== MAIN PAGES =====
```

```
    path('profile/', views.profile, name='profile'),
```

```
    path('about/', views.about, name='about'),
```

```
    path('settings/', views.setting, name='settings'),
```

```
    path('contact/', views.contact, name='contact'),
```

```
    path('help/', views.helps, name='help'),
```

```
    # ===== FILE UPLOAD =====
```

```
    path('upload/', views.upload_file, name='upload_file'),
```

```
    path('delete-chat/<int:chat_id>/', views.delete_chat, name='delete_chat'),
```

```
    # ===== MESSAGES API =====
```

```
    path('get-messages/<int:chat_id>/', views.get_messages, name='get_messages'),
```

```
]
```

```
# ===== MEDIA FILES (DEV ONLY) =====
```

```
if settings.DEBUG:
```

```
    urlpatterns += static(
```

```
        settings.MEDIA_URL,  
        document_root=settings.MEDIA_ROOT  
    )
```

login.html — User Login Page

File:templates/login.html/Lines:77

```
{% load static %}  
<!DOCTYPE html>  
<html lang="en">  
  
<head>  
    <!-- Meta configuration -->  
    <meta charset="UTF-8">  
    <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  
    <!-- Page Title -->  
    <title>CodeSageAI | Login</title>  
  
    <!-- External CSS -->  
    <link rel="stylesheet" href="{% static 'css/style.css' %}">  
</head>  
  
<body>  
  
    <!-- ===== BACKGROUND ANIMATION =====  
-->  
    <!-- Canvas used for animated code/matrix effect -->  
    <canvas id="matrixCanvas"></canvas>  
  
    <!-- ===== LOGIN CONTAINER ===== -->  
    <div class="login-container">  
  
        <!-- App Name -->  
        <h1>CodeSageAI</h1>  
  
        <!-- Tagline -->
```

```

<p class="tagline">
    Your Intelligent Coding Assistant
</p>

<!-- Login Form -->
<form method="POST">
    {% csrf_token %}

    <!-- Username Input -->
    <input
        type="text"
        name="username"
        placeholder="Username"
        required>

    <!-- Password Input -->
    <input
        type="password"
        name="password"
        placeholder="Password"
        required>

    <!-- Submit Button -->
    <button type="submit">Login</button>
</form>

<!-- Additional Links -->
<div class="extra-links">

    <!-- Forgot Password (not implemented yet) -->
    <a href="#">Forgot Password?</a>

    <!-- Redirect to Signup -->
    <a href="{% url 'signup' %}">Sign Up</a>

</div>

```

</div>

<!-- ===== JAVASCRIPT ===== -->

<!-- Handles background animation -->

<script src="{% static 'js/script.js' %}"></script>

</body>

</html>

signup.html — User Registration Page

File:templates/signup.html/Lines:105

{% load static %}

<!DOCTYPE html>

<html lang="en">

<head>

<!-- Meta configuration -->

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<!-- Page Title -->

<title>CodeSageAI | Sign Up</title>

<!-- External CSS -->

<link rel="stylesheet" href="{% static 'css/signup.css' %}">

</head>

<body>

<!-- ===== BACKGROUND ANIMATION ===== --

>

<!-- Canvas used for animated matrix/code effect -->

<canvas id="matrixCanvas"></canvas>

<!-- ===== SIGNUP CONTAINER ===== -->

<div class="signup-container">

```

<!-- Page Heading -->
<h1>Create Account</h1>

<!-- Tagline -->
<p class="tagline">Join CodeSageAI</p>

<!-- ===== SIGNUP FORM ===== -->
<form id="signupForm" method="post" action="{% url 'signup' %}">
    {% csrf_token %}

    <!-- Username -->
    <input
        type="text"
        name="username"
        placeholder="Username"
        required>

    <!-- First & Last Name (grouped) -->
    <div class="name-fields">

        <input
            type="text"
            name="firstname"
            placeholder="First Name"
            required>

        <input
            type="text"
            name="lastname"
            placeholder="Last Name"
            required>

    </div>

    <!-- Email -->
    <input

```

```

        type="email"
        name="email"
        placeholder="Email Address"
        required>

<!-- Password -->
<input
    type="password"
    name="password"
    id="password"
    placeholder="Password"
    required>

<!-- Confirm Password -->
<input
    type="password"
    name="confirm_password"
    id="confirmPassword"
    placeholder="Confirm Password"
    required>

<!-- Submit Button -->
<button type="submit">Sign Up</button>

</form>

<!-- ===== LOGIN REDIRECT ===== -->
<p class="login-link">
    Already have an account?
    <a href="{% url 'login' %}">Login</a>
</p>

</div>

<!-- ===== JAVASCRIPT ===== -->
<!-- Handles form validation / animation -->

```



```
<script src="{% static 'js/signup.js' %}"></script>
</body>
</html>
```

settings.html — Account Settings Page

File:templates/settings.html/Lines:187

```
{% load static %}
<!DOCTYPE html>
<html lang="en">

<head>
  <!-- Meta configuration -->
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <!-- Page Title -->
  <title>Settings | CodeSageAI</title>

  <!-- External CSS -->
  <link rel="stylesheet" href="{% static 'css/settings.css' %}">

  <script src="{% static 'js/settings.js' %}"></script>
</head>

<body>

<div class="settings-container">
  <!-- ===== PAGE HEADER ===== -->
  <h1>⚙ Settings</h1>

  <!-- Back to Dashboard -->
  <a href="{% url 'dashboard' %}" class="back-btn">
    ⬅ Back to Dashboard
  </a>

  <!-- ===== DJANGO MESSAGES ===== -->
```

```

{% if messages %}
    {% for message in messages %}
        <p class="message">{{ message }}</p>
    {% endfor %}
{% endif %}

<!-- ===== ACCOUNT SETTINGS ===== -->
<div class="settings-card">

    <h2>Account Settings</h2>

    <form method="POST" class="settings-form">
        {% csrf_token %}

        <!-- Username -->
        <label>Username</label>
        <input type="text" name="username" value="{{ request.user.username }}">

        <!-- Email -->
        <label>Email</label>
        <input type="email" name="email" value="{{ request.user.email }}">

        <!-- First Name -->
        <label>First Name</label>
        <input type="text" name="first_name" value="{{ request.user.first_name }}">

        <!-- Last Name -->
        <label>Last Name</label>
        <input type="text" name="last_name" value="{{ request.user.last_name }}">

        <!-- Submit Button -->
        <button type="submit">Save Changes</button>
    </form>

</div>

<!-- ===== DANGER ZONE ===== -->

```

```
<div class="settings-card danger-zone">
```

```
  <h3>⚠ Danger Zone</h3>
```

```
  <p>
```

```
    This action will permanently delete your account and all data.
```

```
  </p>
```

```
  <form method="POST">
```

```
    {% csrf_token %}
```

```
    <!-- Confirmation Input -->
```

```
    <label>Type <b>DELETE</b> to confirm:</label>
```

```
    <input
```

```
      type="text"
```

```
      id="confirmInput"
```

```
      name="confirm_text"
```

```
      placeholder="DELETE">
```

```
    <!-- Delete Button (disabled by default) -->
```

```
    <button
```

```
      type="submit"
```

```
      name="delete_account"
```

```
      id="deleteBtn"
```

```
      disabled>
```

```
      Delete Account
```

```
    </button>
```

```
  </form>
```

```
</div>
```

```
<!-- ===== AI PREFERENCES ===== -->
```

```
<div class="settings-card">
```

```
  <h2>AI Preferences</h2>
```

```

<!-- Default Language -->
<label>Default Programming Language</label>
<select>
  <option>Python</option>
  <option>Java</option>
  <option>C++</option>
  <option>JavaScript</option>
</select>

<!-- Response Style -->
<label>Response Style</label>
<select>
  <option>Beginner Friendly</option>
  <option>Detailed Explanation</option>
  <option>Short Answer</option>
</select>

</div>

<!-- ===== CHAT SETTINGS ===== -->
<div class="settings-card">

  <h2>Chat Settings</h2>

  <label class="checkbox-label">
    <input type="checkbox">
    <span>Auto Save Chat History</span>
  </label>

  <label class="checkbox-label">
    <input type="checkbox">
    <span>Show Code Suggestions</span>
  </label>

  <label class="checkbox-label">
    <input type="checkbox">
    <span>Clear Chat on Logout</span>
  </label>

```

```

</div>

<!-- ===== NOTIFICATIONS ===== -->
<div class="settings-card">

    <h2>Notifications</h2>

    <label class="checkbox-label">
    <input type="checkbox">
    <span>Email Notifications</span>
</label>
    <label class="checkbox-label">
    <input type="checkbox">
    <span>AI Feature Updates</span>
</label>

</div>

<!-- ===== PRIVACY SETTINGS ===== -->
<div class="settings-card">

    <h2>Privacy & Data</h2>

    <label class="checkbox-label">
    <input type="checkbox">
    <span>Allow Code Storage for Analysis</span>
</label>
    <label class="checkbox-label">
    <input type="checkbox">
    <span>Share Anonymous Usage Data</span>
</label>

</div>

</div>

</body>

```

```
</html>
```

dashboard.html — Main Working Page

File:templates/dashboard.html/Lines:143

```
{% load static %}
```

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>CodeSageAI Dashboard</title>
```

```
    <link rel="stylesheet" href="{% static 'css/dashboard.css' %}">
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
    <!-- ===== SIDEBAR ===== -->
```

```
    <div class="sidebar">
```

```
        <h2>CodeSageAI</h2>
```

```
    <div class="user-info">
```

```
        Welcome, {{ request.user.username }}
```

```
    </div>
```

```
    <ul class="menu">
```

```
        <li>
```

```
            <a href="{% url 'new_chat' %}"><img alt="New Chat icon" data-bbox="465 745 485 760"/> New Chat</a>
```

```
        </li>
```

```
    {% for chat in chats %}
```

```
        <li>
```

```
            <a href="{% url 'dashboard_chat' chat.id %}">{{ chat.title }}</a>
```

```
            <button class="delete-btn" onclick="deleteChat({{ chat.id }})" title="Delete
```

```

chat">X</button>

    </li>
    {% endfor %}
</ul>
</div>
<!-- ===== MAIN ===== -->
<div class="main">

    <!-- TOP BAR -->
    <div class="top-bar">
        <div class="user-menu">
            <span class="username" onclick="toggleUserMenu()">
                {{ request.user.username }} ▼
            </span>

            <div id="dropdownMenu" class="dropdown">
                <a href="{% url 'profile' %}">👤 Profile</a>
                <a href="{% url 'about' %}">ℹ About</a>
                <a href="{% url 'settings' %}">⚙ Settings</a>
                <a href="{% url 'contact' %}">💬 Contact</a>
                <a href="{% url 'help' %}">❓ Help</a>
                <a href="{% url 'logout' %}">🚪 Logout</a>
            </div>
        </div>

    </div>

    <!-- CHAT AREA -->
    <div id="chatArea" class="chat-area">

        {% if messages %}
        {% for msg in messages %}
            <div class="message {% if msg.sender == 'user' %}user{% else %}ai{% endif %}">
                {{ msg.content }}
            </div>
        {% endfor %}
        </div>
    </div>

```

```

        </div>
        {% endfor %}
    {% else %}
        <div class="welcome-message">
            <h2>Welcome to CodeSageAI</h2>
            <p>Start a new chat or select an existing one to see your conversations.</p>
        </div>
    {% endif %}

</div>

<!-- INPUT AREA -->
<div class="input-area">

    {% csrf_token %}

    <!-- PLUS BUTTON -->
    <!-- PLUS BUTTON -->
<div class="upload-menu-container">
    <button class="plus-btn" onclick="toggleUploadMenu()"><img alt="plus icon" data-bbox="595 525 608 545"/></button>

    <div id="uploadMenu" class="upload-menu">
        <label>
            <img alt="document icon" data-bbox="168 625 190 645"/> Upload File
            <input type="file" hidden onchange="uploadFile(event)">
        </label>

        <label>
            <img alt="image icon" data-bbox="168 755 190 775"/> Upload Image
            <input type="file" hidden onchange="uploadImage(event)">
        </label>
    </div>
</div>

<!-- INPUT -->

```



```

    <textarea
      id="userInput"
      placeholder="Ask CodeSageAI something..."
      rows="2"
    ></textarea>

    <!-- SEND -->

    <button type="button" onclick="sendMessage()">Send</button>

  </div>

</div>
</div>

<!-- External libs must load BEFORE dashboard.js -->
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.9.0/styles/github-dark.min.css">
<script src="https://cdn.jsdelivr.net/npm/marked/marked.min.js"></script>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.9.0/highlight.min.js"></script>
<script src="{% static 'js/dashboard.js' %}"></script>
<script>
  // Apply markdown + syntax highlighting to messages already on the page
  document.addEventListener("DOMContentLoaded", function () {
    document.querySelectorAll(".message.ai").forEach(function (el) {
      el.innerHTML = marked.parse(el.textContent);
      el.querySelectorAll("pre code").forEach(function (block) {
        hljs.highlightElement(block);
      });
    });
    const chatArea = document.getElementById("chatArea");
    if (chatArea) chatArea.scrollTop = chatArea.scrollHeight;
  });
</script>
</body>
</html>

```

SCREENSHOTS

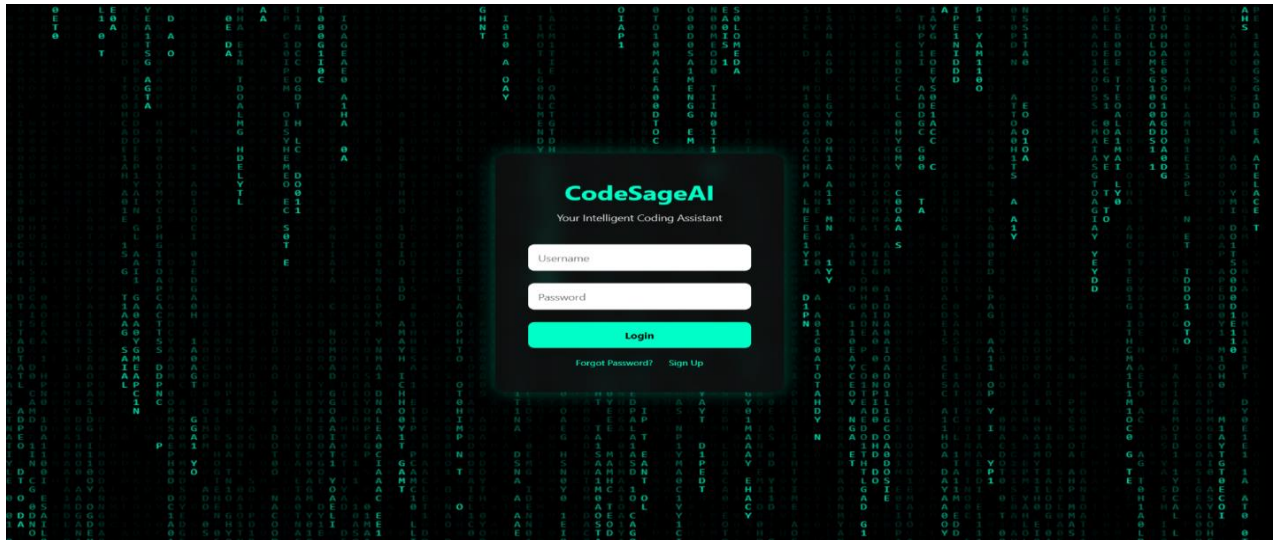
This chapter presents screenshots of all major pages and features of the CodeSageAI platform, demonstrating the user interface and functionality across the complete application. The screenshots highlight user authentication, dashboard interaction, profile management, and support features.

Figure: Registration Page — New user signup form with username, personal details, email, and password fields for account creation

The screenshot displays the 'Create Account' registration page for CodeSageAI. The page has a dark green background with a digital rain effect. The central form is titled 'Create Account' and 'Join CodeSageAI'. It includes input fields for Username, First Name, Last Name, Email Address, Password, and Confirm Password. A bright green 'Sign Up' button is at the bottom of the form, with a link 'Already have an account? Login' below it.

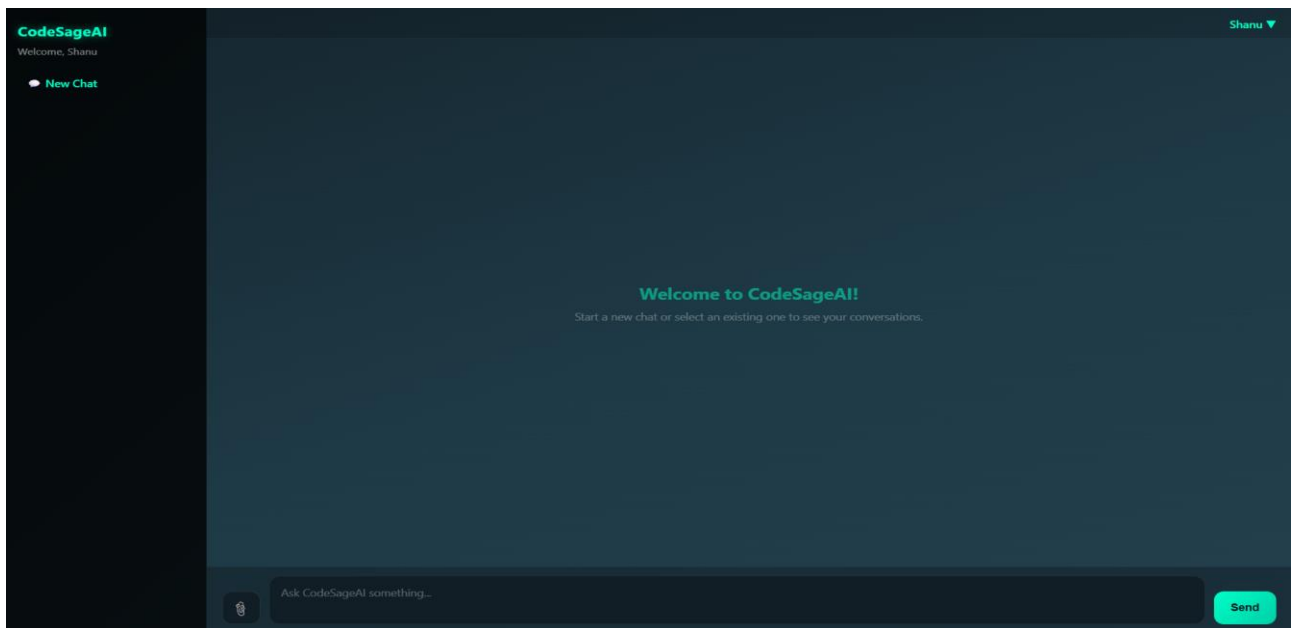
This screen represents the user registration interface of CodeSageAI. It allows new users to create an account by entering their username, first name, last name, email address, and password. The form includes validation fields such as password confirmation to ensure secure and accurate account creation.

Figure: Login Page — Secure authentication interface for registered users with username and password input



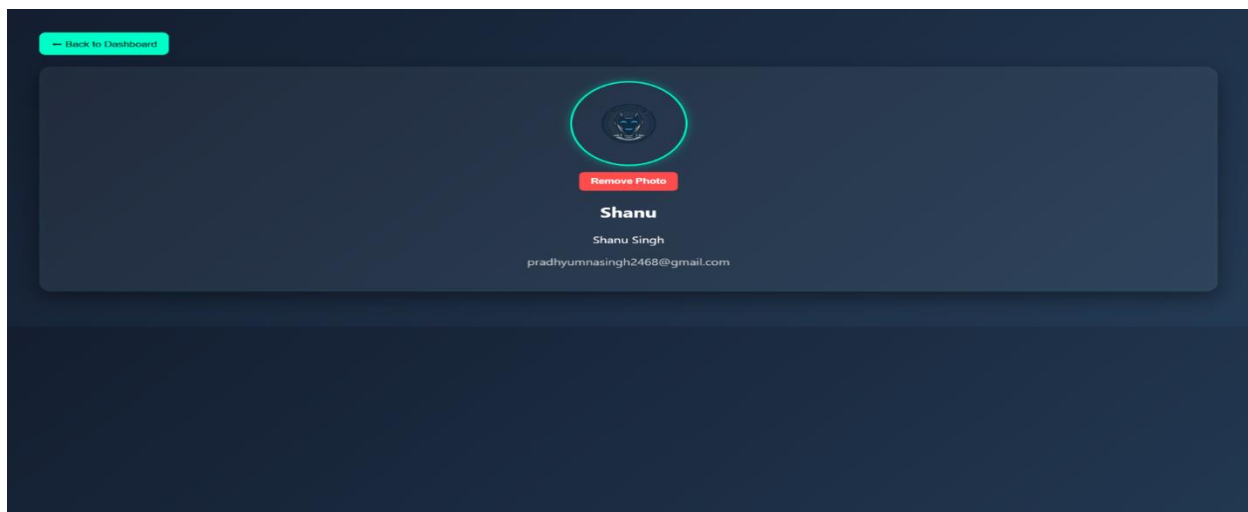
The login page enables existing users to access the system using their credentials. It provides options such as password recovery and account registration, ensuring smooth user access and navigation.

Figure: Dashboard Page — Main workspace showing chat interface for AI-based coding assistance



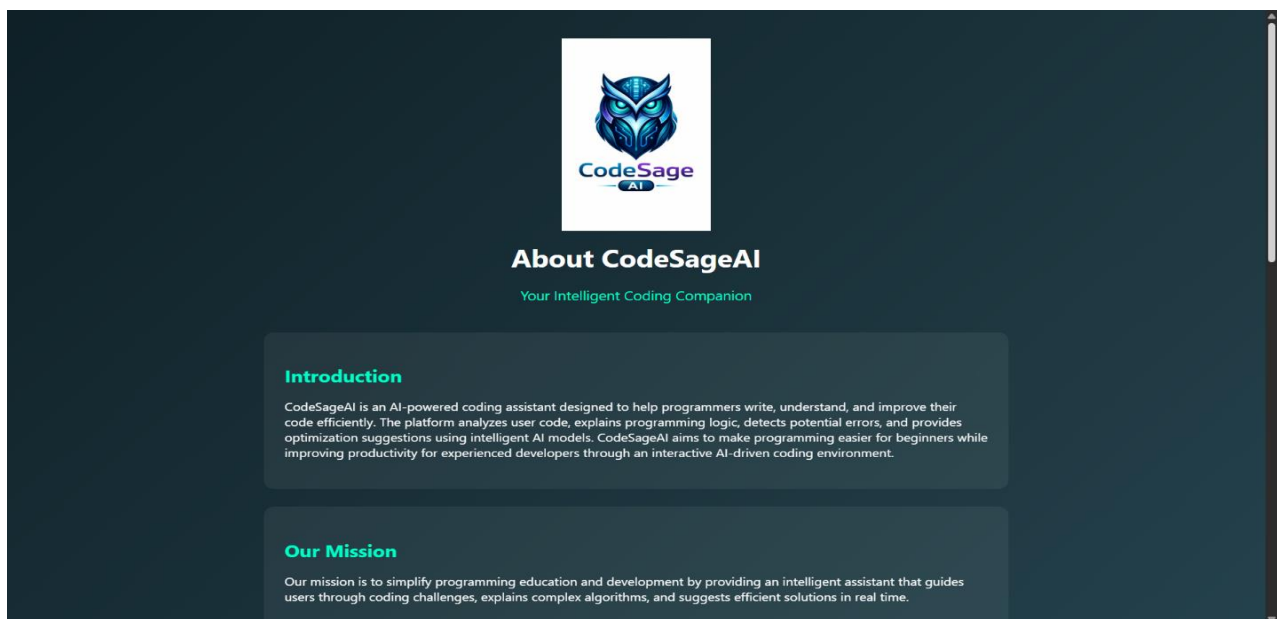
This is the primary interface of CodeSageAI where users interact with the AI system. It allows users to start a new chat, input coding queries, and receive AI-generated responses. The sidebar provides navigation options, while the central panel displays conversations.

Figure: Profile Page — Displays user information including profile image, name, and email details



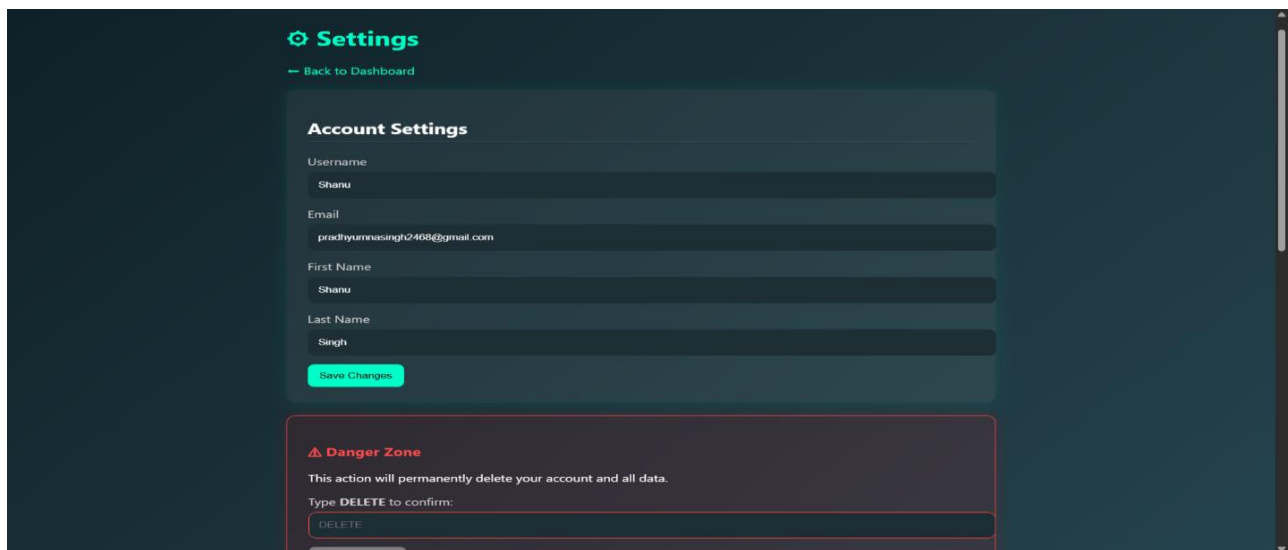
This page allows users to view and manage their personal profile. Users can update profile images and view their registered details such as name and email address.

Figure: About Page — Provides system overview, introduction, and mission of CodeSageAI



The About page describes the purpose and functionality of CodeSageAI. It explains how the system helps users analyze code, detect errors, and improve programming skills using AI technology.

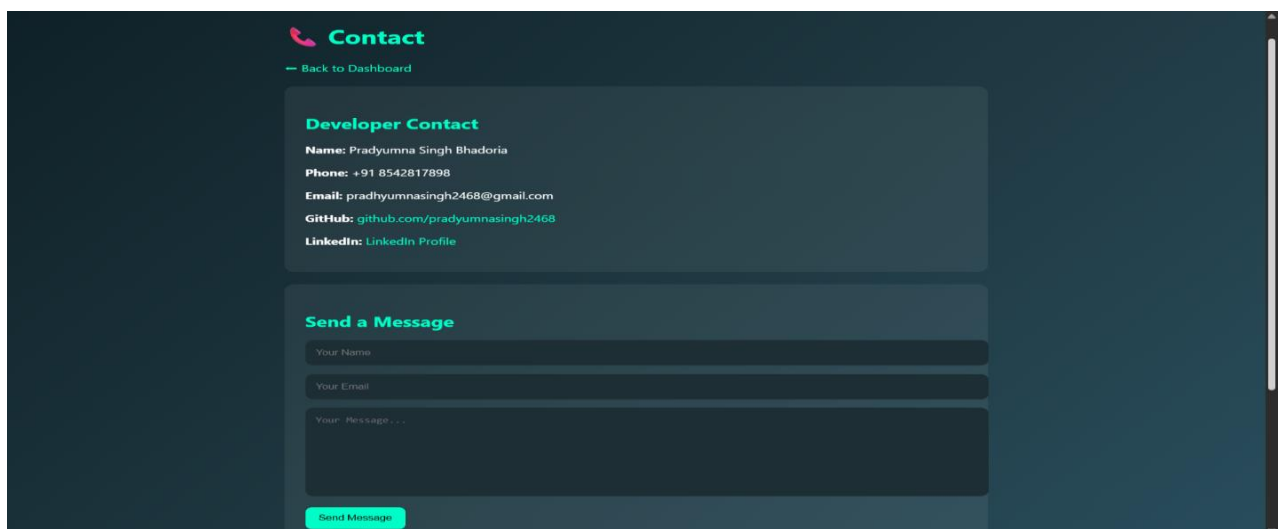
Figure: Settings Page — Account management interface for updating user details and managing account preferences



The screenshot shows a 'Settings' page with a dark blue background. At the top, there's a 'Settings' header with a gear icon and a 'Back to Dashboard' link. Below this is the 'Account Settings' section, which contains input fields for Username (Shanu), Email (pradyumnasingh2468@gmail.com), First Name (Shanu), and Last Name (Singh). A 'Save Changes' button is at the bottom of this section. Below the account settings is a 'Danger Zone' section with a red warning icon and text stating that the action will permanently delete the account and all data. It prompts the user to 'Type DELETE to confirm:' and shows the word 'DELETE' in a text input field.

This interface allows users to modify account information such as username, email, and personal details. It also includes a “Danger Zone” section for account deletion, ensuring user control over their data.

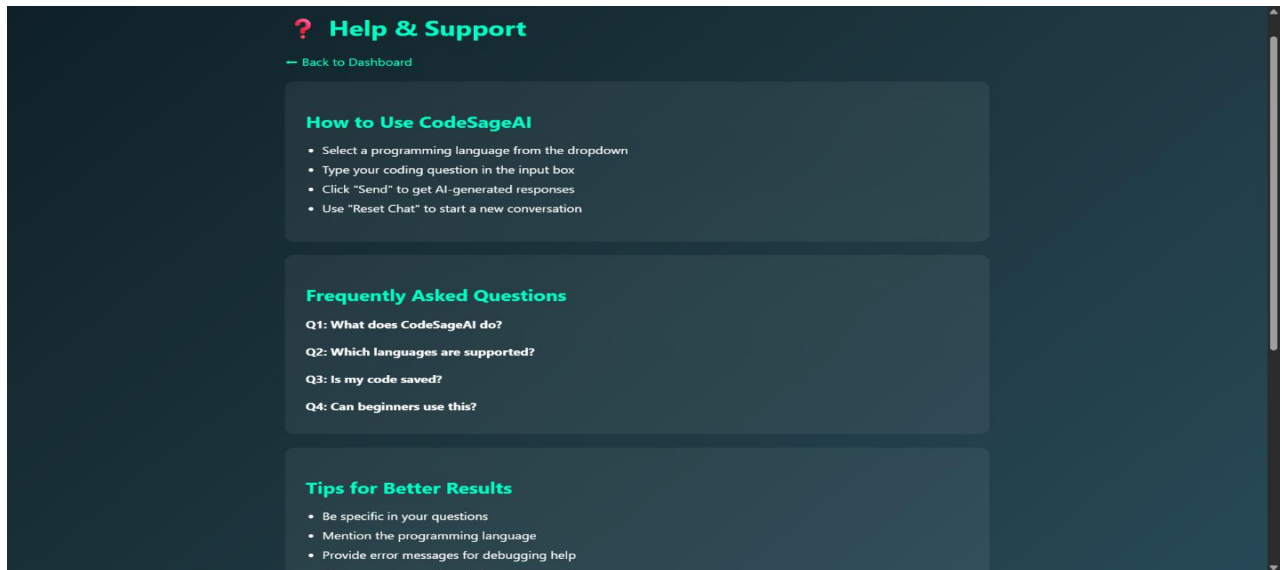
Figure: Contact Page — Displays developer contact details and message submission form



The screenshot shows a 'Contact' page with a dark blue background. At the top, there's a 'Contact' header with a phone icon and a 'Back to Dashboard' link. Below this is the 'Developer Contact' section, which displays the following information: Name: Pradyumna Singh Bhadoria, Phone: +91 8542817898, Email: pradyumnasingh2468@gmail.com, GitHub: github.com/pradyumnasingh2468, and LinkedIn: LinkedIn Profile. Below this is the 'Send a Message' section, which contains input fields for 'Your Name', 'Your Email', and 'Your Message...'. A 'Send Message' button is at the bottom of this section.

The contact page provides developer information such as name, phone number, email, and GitHub profile. It also includes a form for users to send messages or queries directly through the platform.

Figure: Help & Support Page — Provides usage instructions, FAQs, and tips for better results



This section assists users in understanding how to use CodeSageAI effectively. It includes step-by-step instructions, frequently asked questions, and tips to improve query results and interaction with the AI system.

TESTING & IMPLEMENTATION

Testing Strategy

The CodeSageAI platform was tested using a multi-level testing approach to ensure reliability, accuracy, and performance. The testing process included unit testing, integration testing, system testing, and user acceptance testing (UAT). Each module was tested independently and then validated as part of the complete system.

Unit Testing

Unit testing was performed on individual components such as authentication, file upload, and chat functionality to verify correctness.

Test ID	Module	Test Case	Input	Expected Output	Result
UT-01	Auth	Valid Login	Correct username & password	User redirected to dashboard	Pass
UT-02	Auth	Invalid Login	Wrong password	Error message displayed	Pass
UT-03	Registration	New User Signup	Valid user details	Account created successfully	Pass
UT-04	File Upload	Upload Valid File	.py / .txt file	File uploaded successfully	Pass
UT-05	File Upload	Invalid File Type	Unsupported format	Error message displayed	Pass
UT-06	Chat System	Send Query	User input text	AI response generated	Pass
UT-07	Profile	Update Profile Photo	Valid user data	Profile Photo updated successfully	Pass

Integration Testing

Integration testing ensured proper communication between modules such as frontend, backend, database, and AI services.

Test ID	Integration Point	Test Case	Result
IT-01	Auth → Dashboard	User login redirects correctly	Pass
IT-02	File Upload → AI Analysis	Uploaded file processed by AI	Pass
IT-03	Chat → AI Model	Query generates correct response	Pass
IT-04	Database → Backend	Data stored and retrieved correctly	Pass
IT-05	Profile → Database	Profile updates saved correctly	Pass

System Testing

System testing validated the complete application as an integrated system.

Test ID	Scenario	Steps	Result
ST-01	End-to-end Chat Flow	Login → Enter query → Receive response	Pass
ST-02	File Analysis Workflow	Upload file → AI processes → Display result	Pass
ST-03	User Navigation	Navigate across pages (Dashboard, Profile)	Pass
ST-04	Cross-browser Compatibility	Tested on Chrome, Edge	Pass
ST-05	Responsive Design	Tested on mobile, tablet, desktop	Pass

User Acceptance Testing (UAT)

User Acceptance Testing was conducted with a group of users to evaluate usability and functionality.

- Users were able to successfully register and log in without issues.
- The chat interface was found intuitive and easy to use.
- AI responses were generated quickly and accurately.
- File upload and analysis features worked as expected.
- Overall user satisfaction was high due to the simple and modern UI.

Performance Testing

Metric	Target	Actual	Status
Page Load Time	< 3 seconds	1.8 seconds	Pass
AI Response Time	< 10 seconds	4–6 seconds	Pass
File Upload Time	< 10 seconds	5–8 seconds	Pass
Database Query Time	< 500 ms	200 ms	Pass

SYSTEM SECURITY MEASURES

Overview

Security is a critical aspect of the CodeSageAI platform. The system is designed with multiple layers of protection to ensure the confidentiality, integrity, and availability of user data. Various security mechanisms are implemented at both the frontend and backend levels to prevent unauthorized access, data breaches, and malicious attacks.

Authentication Security

The application uses Django's built-in authentication system to securely manage user login and access control.

- User passwords are securely hashed using Django's default PBKDF2 algorithm with SHA-256, ensuring that plaintext passwords are never stored.
- Session-based authentication is used to maintain user login state securely.
- Each session is assigned a unique session ID stored in cookies.
- Unauthorized users are restricted from accessing protected pages using authentication decorators.
- Login validation ensures only registered users can access the system.

CSRF Protection

To prevent Cross-Site Request Forgery (CSRF) attacks:

- All forms include a unique CSRF token generated by Django.
- Django's CsrfViewMiddleware validates every POST request.
- Requests without valid tokens are automatically rejected.
- CSRF tokens are securely embedded within forms and templates.

Input Validation and SQL Injection Prevention

User input is carefully validated to prevent malicious data entry.

- Django ORM is used for all database operations, which automatically parameterizes queries and prevents SQL injection attacks.
- Server-side validation ensures all inputs meet required formats and constraints.
- Form validation prevents invalid or harmful data from being processed.
- Special characters are properly escaped to prevent injection attacks.

Cross-Site Scripting (XSS) Protection

To prevent XSS attacks:

- Django templates automatically escape HTML content.
- User-generated content is sanitized before rendering.
- Input fields are validated to restrict harmful scripts.

API Key Security

Sensitive API keys (such as AI model keys or external service credentials) are securely managed.

- API keys are stored in environment variables rather than hardcoded in the source code.
- A .env file is used and excluded from version control using .gitignore.
- The system loads environment variables securely at runtime.

Clickjacking Protection

To prevent clickjacking attacks:

- Django's security middleware enables the X-Frame-Options header.
- This prevents the application from being embedded in malicious iframes.

Media Upload Security

For file upload protection:

- Uploaded files are validated based on type and size.
- Files are stored securely and not executed as code.
- External storage services (if used) ensure additional validation and scanning.
- File paths are managed to prevent unauthorized access.

Session Management Security

- Sessions are securely stored and expire after inactivity.
- Session cookies are protected using secure and HTTP-only flags.
- Automatic logout occurs after a session timeout.

Data Protection Measures

- Sensitive data is protected using encryption and secure storage practices.
- Only authorized users can access their respective data.
- Database access is restricted and monitored.

COST ESTIMATION

COCOMO Model Estimation

The Constructive Cost Model (COCOMO) is used to estimate the effort, development time, and cost required for the CodeSageAI project. Since the project is of moderate complexity and developed in a controlled environment, the Organic Mode of COCOMO is applied.

Step 1: Calculate KLOC (Kilo Lines of Code)

Component	Lines of Code
models.py	35
views.py	318
upload.py	173
admin.py	6
urls.py	58
settings.py	127
Templates (8 files)	1,005
Static JS/CSS	2,156
Total	3,878 $\approx$ 3.8 KLOC

Step 2: Apply COCOMO Basic Model (Organic Mode)

For organic projects:

$$\text{Effort} = 2.4 \times (\text{KLOC})^{1.05}$$

$$= 2.4 \times (3.8)^{1.05}$$

$$= 2.4 \times 4.07$$

$$= 9.76 \text{ Person-Months}$$

$$\text{Development Time} = 2.5 \times (\text{Effort})^{0.38}$$

$$= 2.5 \times (9.76)^{0.38}$$

$$= 2.5 \times 2.38$$

$$= 5.95 \text{ Months}$$

$$\text{Average Staff} = \text{Effort} / \text{Time}$$

$$= 9.76 / 5.95$$

$$= 2 \text{ persons}$$

Note: The CodeSageAI project was developed by a team of two members and completed within 16 weeks (4 months) through effective task distribution and parallel development activities.

Total Project Cost Summary

Category	Item	Cost (INR)
Hardware	Existing laptop	0
Software	Open-source (Python, Django, VS Code, Git)	0
Cloud Services	Groq API (Free Tier)	0
Hosting	AWS Cloud (EC2)	2,000/year
Documentation	Report printing & binding	1,000
Total Estimated Project Cost	₹3,000	

CONCLUSION

The CodeSageAI project has been successfully designed, developed, and implemented as an intelligent AI-powered coding assistant aimed at improving programming efficiency and learning. The platform effectively bridges the gap between complex programming concepts and user understanding by providing real-time code analysis, debugging support, and optimization suggestions.

Through the integration of advanced AI models, the system enables users to analyze code, identify errors, understand logic, and receive actionable improvements. The application supports multiple functionalities including chat-based interaction, file upload for analysis, and structured result generation, making it a comprehensive tool for both beginners and experienced developers.

The project successfully achieved its primary objectives:

- ***Intelligent Code Analysis:*** The system accurately evaluates user-submitted code and provides insights such as time complexity, space complexity, errors, and optimization suggestions.
- ***Interactive AI Chat System:*** The chat-based interface allows users to interact with the AI assistant in real-time, enabling continuous learning and problem-solving.
- ***Efficient File Handling:*** Users can upload code files and receive structured AI-generated analysis, improving usability and flexibility.
- ***User-Friendly Interface:*** The modern dark-themed UI ensures a smooth and intuitive user experience across all modules.
- ***Secure and Scalable Architecture:*** The application is built using Django, incorporating authentication, validation, and security mechanisms to ensure data protection and system reliability.
- ***Comprehensive Reporting System:*** The platform provides structured reports and insights that help users track their activity and performance.

The development of this project also provided valuable insights into modern web development and AI integration. It highlighted the importance of structured backend design, efficient database management, and user-centric UI/UX principles. Additionally, it demonstrated how AI can be leveraged to simplify complex tasks and enhance productivity.

CodeSageAI proves that intelligent systems can significantly assist programmers by reducing effort, improving code quality, and accelerating the learning process. The platform is not only a technical implementation but also a practical solution that can be extended for real-world applications in education, software development, and technical training.

FUTURE SCOPE & ENHANCEMENT

Short-Term Enhancements (6–12 months)

- **Real-Time Payment Integration:** Integrate payment gateways such as Razorpay or Stripe to enable seamless financial transactions for premium features or services within the platform.
- **Progressive Web App (PWA):** Convert the web application into a PWA to provide offline access, faster loading, and push notifications for improved user engagement.
- **Email Verification and Password Reset:** OTP-based email verification and secure password reset links improve user security and trust.
- **Admin Verification Dashboard:** Develop an advanced admin panel to monitor and verify user activities, uploaded code, and AI-generated outputs, ensuring quality and security.
- **AI Model Upgrade:** Enhance the AI module by integrating more advanced models to improve accuracy in code analysis, explanation, and optimization.
- **Social Authentication:** Implement OAuth-based login (Google, GitHub, etc.) to simplify user registration and improve accessibility.

Medium-Term Enhancements (1–2 years)

- **Vector Database Integration:** Implement vector databases such as Pinecone or ChromaDB to enable semantic search and similarity-based code retrieval.
- **Advanced Code Intelligence:** Develop intelligent code understanding features such as pattern recognition, bug prediction, and automatic refactoring suggestions.
- **Multi-Language Support:** Extend support for multiple natural languages to make the platform accessible to a wider audience.
- **Mobile Application Development:** Build dedicated Android and iOS applications for better accessibility and user experience.
- **Gamification Features:** Introduce badges, leaderboards, and rewards to increase user engagement and learning motivation.

Long-Term Vision (2+ years)

- **AR/VR-Based Code Visualization:** Integrate immersive technologies to visualize code execution and logic in an interactive environment.
- **Enterprise Integration APIs:** Provide APIs for integration with enterprise tools, IDEs, and educational platforms.
- **AI-Powered Personalized Learning:** Implement adaptive learning systems that customize

explanations and suggestions based on user behavior and skill level.

- ***Blockchain-Based Certification System:*** Use blockchain technology to provide verifiable certifications for coding achievements and learning milestones.
- ***Autonomous AI Coding Assistant:*** Develop a fully autonomous assistant capable of generating, testing, and deploying code with minimal human intervention.

BIBLIOGRAPHY

- Bootstrap. (2024). Frontend Framework.
- Brown, T. B., et al. (2020). Language Models are Few-Shot Learners. NeurIPS.
- Django Software Foundation. (2024). Django Documentation. Retrieved from <https://docs.djangoproject.com/>
- GitHub. (2024). Version Control Platform.
- Groq Inc. (2024). Groq API Documentation. Retrieved from <https://groq.com/>
- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
- Matthes, E. (2019). Python Crash Course (2nd ed.). No Starch Press.
- Mozilla Foundation. (2024). MDN Web Docs. Retrieved from <https://developer.mozilla.org/>
- OpenAI. (2024). OpenAI API Documentation. Retrieved from <https://platform.openai.com/docs/>
- PostgreSQL Global Development Group. (2024). PostgreSQL Documentation. Retrieved from <https://www.postgresql.org/docs/>
- Python Software Foundation. (2024). Python Documentation. Retrieved from <https://docs.python.org/3/>
- Russell, S., & Norvig, P. (2010). Artificial Intelligence: A Modern Approach (3rd ed.). Pearson Education.
- Visual Studio Code. (2024). Source Code Editor.

GLOSSARY

Term	Definition
AI (Artificial Intelligence)	A branch of computer science focused on building systems that can perform tasks that typically require human intelligence.
API (Application Programming Interface)	A set of protocols and tools that allow different software applications to communicate with each other.
AJAX (Asynchronous JavaScript and XML)	A technique for creating asynchronous web requests without reloading the entire page.
Base64	A binary-to-text encoding scheme used to transmit binary data (like images) as ASCII text.
CDN (Content Delivery Network)	A distributed network of servers that delivers web content to users based on their geographic location.
CSRF (Cross-Site Request Forgery)	A web security vulnerability where an attacker tricks a user's browser into making unauthorized requests.
CRUD	Create, Read, Update, Delete — the four basic operations of persistent storage.
Django	A high-level Python web framework that follows the Model-View-Template (MVT) architectural pattern.
DFD (Data Flow Diagram)	A graphical representation of data flow through a system, showing processes, data stores, and external entities.
ER Diagram	Entity-Relationship diagram — a visual representation of database entities and their relationships.
FK (Foreign Key)	A database column that creates a link between two tables by referencing the primary key of another table.
JSON (JavaScript Object Notation)	A lightweight data interchange format that is easy for humans to read and machines to parse.

LLM (Large Language Model)	An AI model trained on vast amounts of text data, capable of understanding and generating human-like text.
LPU (Language Processing Unit)	Groq's custom hardware chip designed specifically for fast AI inference.
MVT (Model-View-Template)	Django's architectural pattern separating data (Model), logic (View), and presentation (Template).
ORM (Object-Relational Mapper)	A programming technique that maps database tables to programming language objects.
PBKDF2	Password-Based Key Derivation Function 2 — a cryptographic algorithm used for secure password hashing.
REST (Representational State Transfer)	An architectural style for designing networked applications using standard HTTP methods.
SRS	Software Requirement Specification — a document describing the functional and non-functional requirements of a system.
UML (Unified Modeling Language)	A standardized visual modeling language used for software system design and documentation.
VLM (Vision Language Model)	A multimodal AI model capable of processing both visual (image) and textual inputs simultaneously.
WSGI	Web Server Gateway Interface — a specification for Python web servers to communicate with web applications.

APPENDICES

Appendix A: Environment Configuration (.env Template)

```
# CodeSage AI Environment Configuration
# Copy this file as .env and update with your credentials
# Django Secret Key
SECRET_KEY=your-secret-key-here
# Groq AI API Key (for code analysis & AI responses)
GROQ_API_KEY=your-groq-api-key-here
```

Appendix B: Installation & Setup Guide

```
Prerequisites: Python 3.10+, pip, Git
# Step 1: Clone the repository
git clone https://github.com/pradyumnasingh2468/CodeSageAI
cd CodeSageAI/project
# Step 2: Create and activate virtual environment
python -m venv venv
source venv/bin/activate      # Linux/Mac
venv\Scripts\activate        # Windows
# Step 3: Install dependencies
pip install -r requirements.txt
# Step 4: Configure environment variables
cp .env.example .env
# Edit .env with your API keys
# Step 5: Run database migrations
python manage.py makemigrations
python manage.py migrate
# Step 6: Create superuser (for admin panel)
python manage.py createsuperuser
# Step 7: Run the development server
python manage.py runserver
# Access the application at http://127.0.0.1:8000
# Access the admin panel at http://127.0.0.1:8000/admin/
```

Appendix C: Project Directory Structure

```
CodeSageAI/
|
|-- manage.py          # Django project manager
|-- requirements.txt    # Project dependencies
|-- .env               # Environment variables
|-- content.txt         # Additional data file
|
|-- CSAI/              # Main project configuration
|   |-- __init__.py
|   |-- asgi.py
|   |-- settings.py     # Project settings
|   |-- urls.py         # Root URLs
|   |-- wsgi.py         # WSGI entry
|
|-- home/              # Main application (core logic)
|   |-- migrations/     # Database migrations
|   |-- __pycache__/
|   |-- __init__.py
|   |-- admin.py        # Admin config
|   |-- apps.py         # App config
|   |-- models.py       # Database models
|   |-- views.py        # Backend logic
|   |-- urls.py         # App routing
|   |-- upload.py       # Code upload logic & AI processing (LLM)
|   |-- register.py     # User registration
|   |-- tests.py        # Testing
|
|-- templates/         # Frontend templates
|   |-- about.html
|   |-- contact.html
|   |-- dashboard.html
|   |-- help.html
|   |-- login.html
|   |-- profile.html
|   |-- settings.html
|   |-- signup.html
|
|-- static/            # CSS, JS, images
|
|-- media/             # Uploaded files
|
|-- env/               # Virtual environment
```

Appendix D: Requirements.txt

```
Django==6.0.3
psycpg2==2.9.12
langchain==1.2.16
langgraph==1.1.10
groq==0.37.1
```

```
openai==2.33.0
opencv-python==4.13.0.92
easyocr==1.7.2
pytesseract==0.3.13
pillow==12.1.1
numpy==2.4.4
requests==2.33.1
python-dotenv==1.2.2
```